

Lab 9-10 Report

Nanoprocessor Design Competition



Binara Herath	–	230244G
Janidu Pabasara	–	230451M
Malindu Bandara	–	230078B
Kalhara Jayathissa	–	230291V

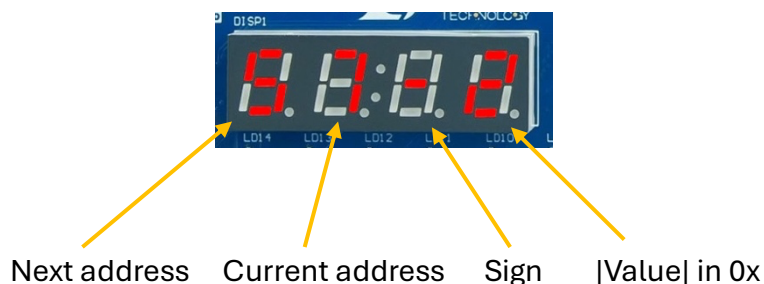
Assigned Task

The objective of this lab was to design and implement a simple 4-bit nanoprocessor that supports four instructions MOVI, ADD, NEG, and JZR. The processor was built using VHDL and tested both in simulation and on the BASYS 3 board. The final goal was to write an assembly program to compute the sum of integers from 1 to 3 and store the result in register 7, with output visible on LEDs and a 7-segment display.

E-nテル Processor Pro : Functional and Visual Upgrades



As an enhancement to the basic nanoprocessor(Base E-nテル Processor), we developed an improved version named the E-nテル Processor Pro. This version includes a user-controlled switch that selects between displaying the value in Register 7 or the output of the Add/Subtract Unit on the output display & LEDs. Furthermore, the rightmost 7-segment digit shows the modulus of the selected value, while the adjacent digit indicates a negative sign if the value is negative. The two leftmost digits of the display show the current and next memory addresses in the Program ROM, enabling real-time tracking of jump instructions and program flow. These enhancements not only make debugging and program analysis easier but also introduce features typically seen in more advanced processors, adding significant educational and functional value to the project.



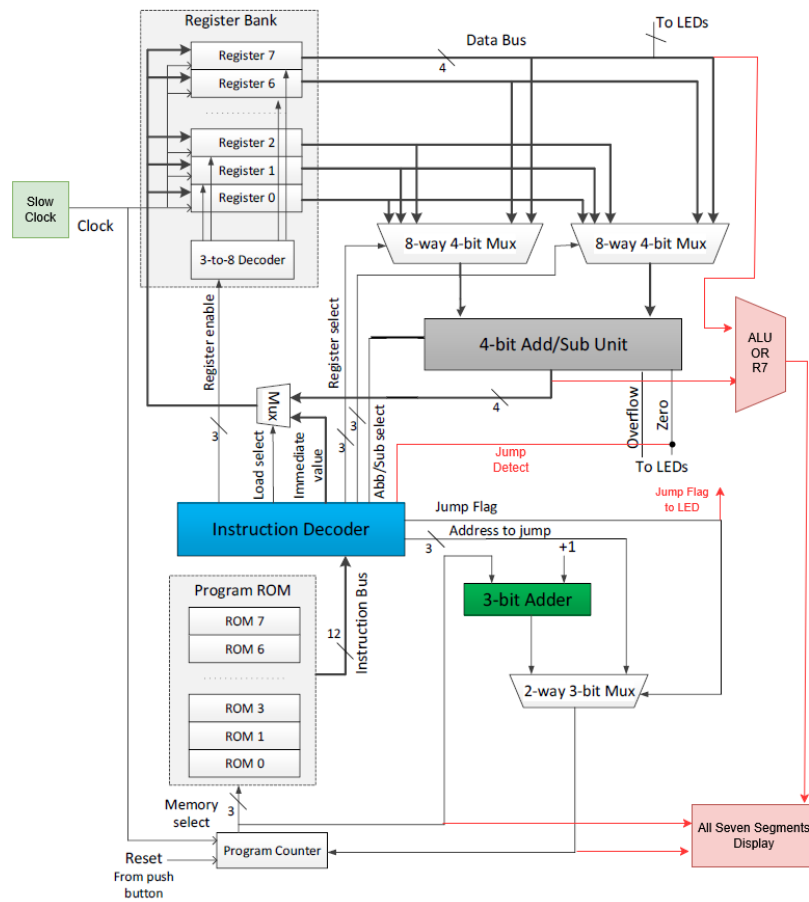
Entity Hierarchy Diagram

```

graph TD
    E_nitel_Processor_Pro[E_nitel_Processor_Pro] --> Display_Display[Display : AllSevenSegment Display]
    E_nitel_Processor_Pro --> Nano_Processor[Nano_Processor]
    E_nitel_Processor_Pro --> MUX_2_Way_4_Bit_Mux[MUX : 2_Way_4_Bit_Mux]
    Display_Display --> LUT_LUT_16_7[LUT : LUT_16_7]
    Display_Display --> Slow_Clock[Slow Clock]
    Nano_Processor --> PC_Increment_Unit[PC Increment Unit]
    Nano_Processor --> Program_ROM[Program ROM]
    Nano_Processor --> Instruction_Decoder[Instruction Decoder]
    Nano_Processor --> ALU_Arithmetic_Unit[ALU : Arithmetic_Unit]
    Nano_Processor --> Register_Bank[Register Bank]
    Nano_Processor --> MUX_2_Way_3_Bit[MUX 2 Way 3 Bit]
    Nano_Processor --> Adder_RCA_3_Bit[Adder : RCA_3_Bit]
    Nano_Processor --> Adder_RCA_4_Bit[Adder : RCA_4_Bit]
    Nano_Processor --> Decoder_3_to_8[Decoder 3 to 8]
    Nano_Processor --> Decoder_2_to_4[Decoder 2 to 4]
    Nano_Processor --> Register[Register]
    PC_Increment_Unit --> Program_Counter[Program Counter]
    Adder_RCA_3_Bit --> FA_Full_Adder_3[FA : Full Adder]
    FA_Full_Adder_3 --> HA_Half_Adder_3[HA : Half Adder]
    Adder_RCA_4_Bit --> FA_Full_Adder_4[FA : Full Adder]
    FA_Full_Adder_4 --> HA_Half_Adder_4[HA : Half Adder]
    Register_Bank --> Register
  
```

The diagram illustrates the entity hierarchy of the **E_nitel_Processor_Pro** project. The root entity is **E_nitel_Processor_Pro**, which branches into three main components: **Display : AllSevenSegment Display**, **Nano_Processor**, and **MUX : 2_Way_4_Bit_Mux**.

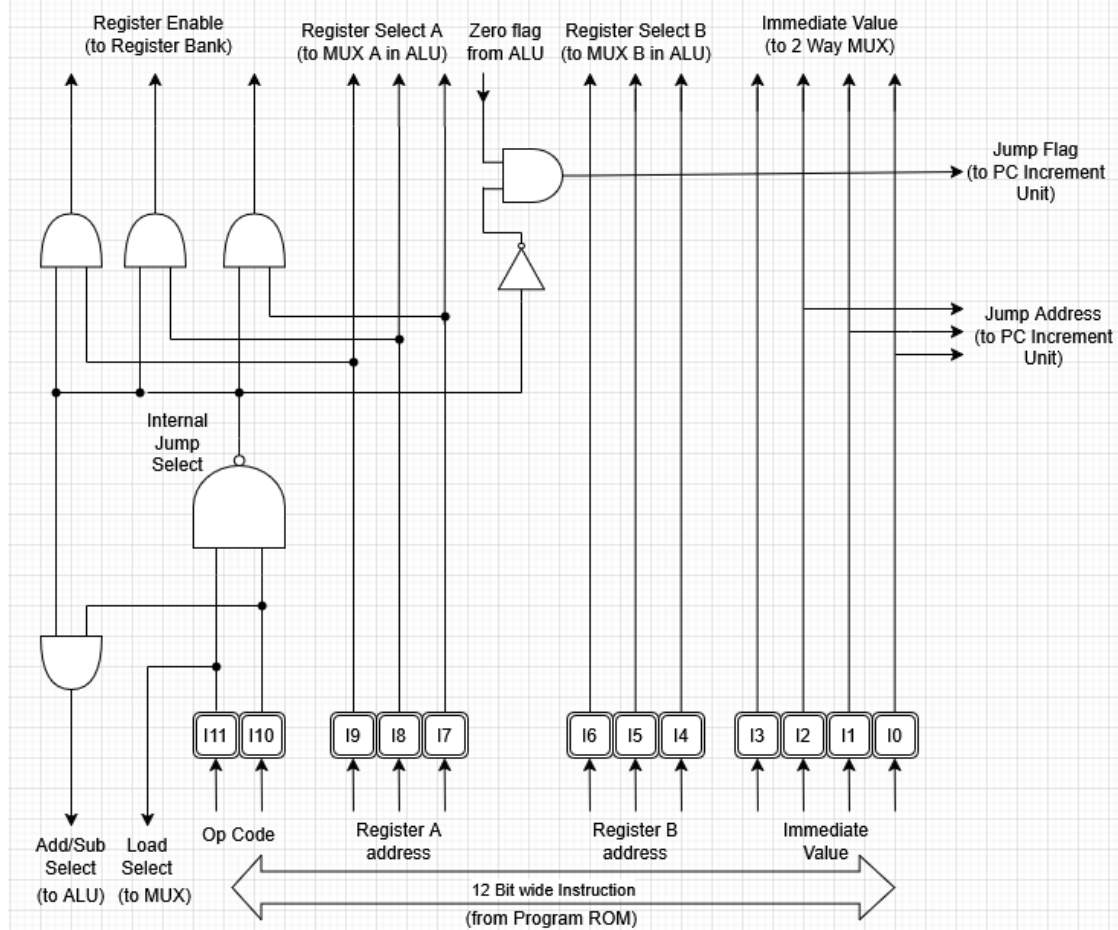
- Display : AllSevenSegment Display**
 - LUT : LUT_16_7
 - Slow Clock
- Nano_Processor**
 - PC Increment Unit
 - Program Counter
 - Program ROM
 - Instruction Decoder
 - ALU : Arithmetic_Unit
 - Adder Subtractor
 - Adder : RCA_4_Bit
 - FA : Full Adder
 - HA : Half Adder
 - 2 Way 4 Bit MUX
 - Decoder 3 to 8
 - Decoder 2 to 4
 - Register Bank
 - Register
 - MUX 2 Way 3 Bit
 - Adder : RCA_3_Bit
 - FA : Full Adder
 - HA : Half Adder



Description: Instruction Decoder

The Instruction Decoder module interprets a 12-bit instruction fetched from the program ROM and generates control signals required by various processor components such as the ALU, register bank, MUXes, and the program counter (PC) unit. It uses the opcode and other fields in the instruction to determine the required control logic.

Gate-Level Logic Diagram for Instruction Decoder



Inputs:

Instruction (11 downto 0):

A 12-bit instruction word divided into:

Instruction(11 downto 10) → Opcode

Instruction(9 downto 7) → Register A address

Instruction(6 downto 4) → Register B address

Instruction(3 downto 0) → Immediate value

Zero_Flag:

Status flag output from the ALU indicating whether the result is zero (used for conditional jump decisions).

Outputs:

RegSelA (2:0):

Register select A for ALU input MUX A (from bits 9–7 of the instruction).

RegSelB (2:0):

Register select B for ALU input MUX B (from bits 6–4 of the instruction).

Reg_Enb (2:0):

Register enable signal to the register bank. Enables writing into a register if the instruction is not a jump. Derived from bits 9–7 and a jump-detection condition.

Immediate (3:0):

Immediate value used by MOVI instruction and passed to a MUX (from bits 3–0 of the instruction).

Load_Sel:

Select line to a 2-way MUX deciding between using the Immediate value or a register value for ALU input.

1 for immediate instructions (MOVI, JUMP)

0 for register-based operations (ADD, NEG)

AddSub:

ALU control signal that chooses between addition and subtraction.

0 → Add

1 → Subtract (used only in NEG instruction)

JumpAddr (2:0):

Address to jump to (from bits 2–0) in case of a conditional jump (JZR instruction).

Jump_Flag:

Signal to control the PC increment logic. Activated when the opcode is JUMP and the Zero_Flag is 1.

Internal Logic Explained:

Load_Sel = Instruction(11)

Directly derived from the most significant bit of the opcode to identify immediate operations.

Jump detection logic (jump_detect_internal):

jump_detect_internal = Instruction(11) NAND Instruction(10)

This internal signal identifies JUMP-type instructions based on the opcode.

Reg_Enb_internal (2:0):

Each bit is the AND of jump_detect_internal with respective bits of RegSelA (Instruction 9–7). If it's a jump, these become 0, disabling write-back.

AddSub = jump_detect_internal AND Instruction(10)

Only the NEG instruction (opcode "01") activates subtraction.

Jump_Flag = NOT jump_detect_internal AND Zero_Flag

Ensures that the jump only occurs when the instruction is indeed a JZR (00) and the ALU result was zero.

Diagram Overview (Draw.io file):

The gate-level diagram visually maps how each part of the instruction connects to gates and control signals:

AND, NAND, and NOT gates are used to implement combinational logic.

Lines from instruction bits are routed to MUX selectors, register enables, and jump flag logic.

The outputs are clearly labeled to show where they go (e.g., “to Register Bank”, “to PC Increment Unit”).

Summary:

This decoder interprets four instruction types:

ADD: Register + Register

NEG: 2's complement of a register

MOVI: Move Immediate

JZR: Jump if Zero (conditional)

It handles control signal generation purely through combinational logic derived from the opcode and other instruction fields.

Assembly Program : asm

```

MOVI R1, 1      ; R1 ← 1
MOVI R2, 2      ; R2 ← 2
ADD R1, R2      ; R1 ← R1 + R2 = 3
MOVI R3, 3      ; R3 ← 3
ADD R1, R3      ; R1 ← R1 + R3 = 6
MOVI R7, 0      ; R7 ← 0
ADD R7, R1      ; R7 ← 6

```

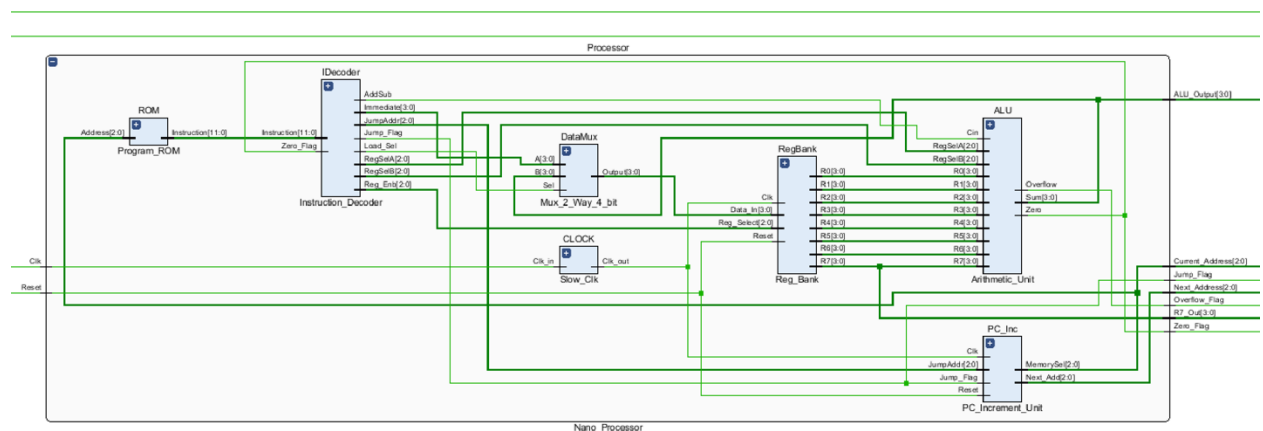
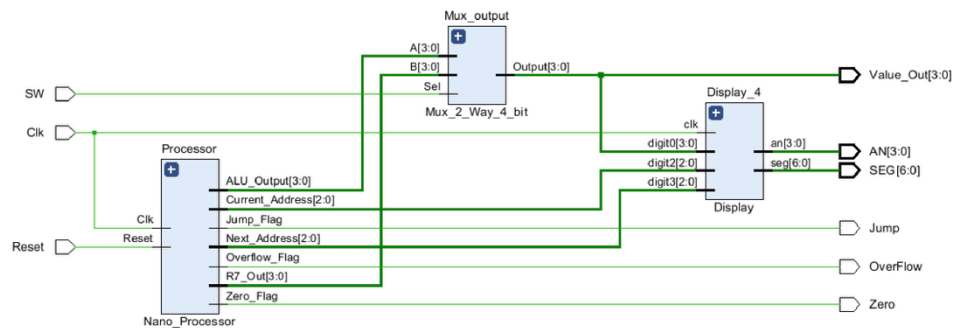
Machine Code : sql

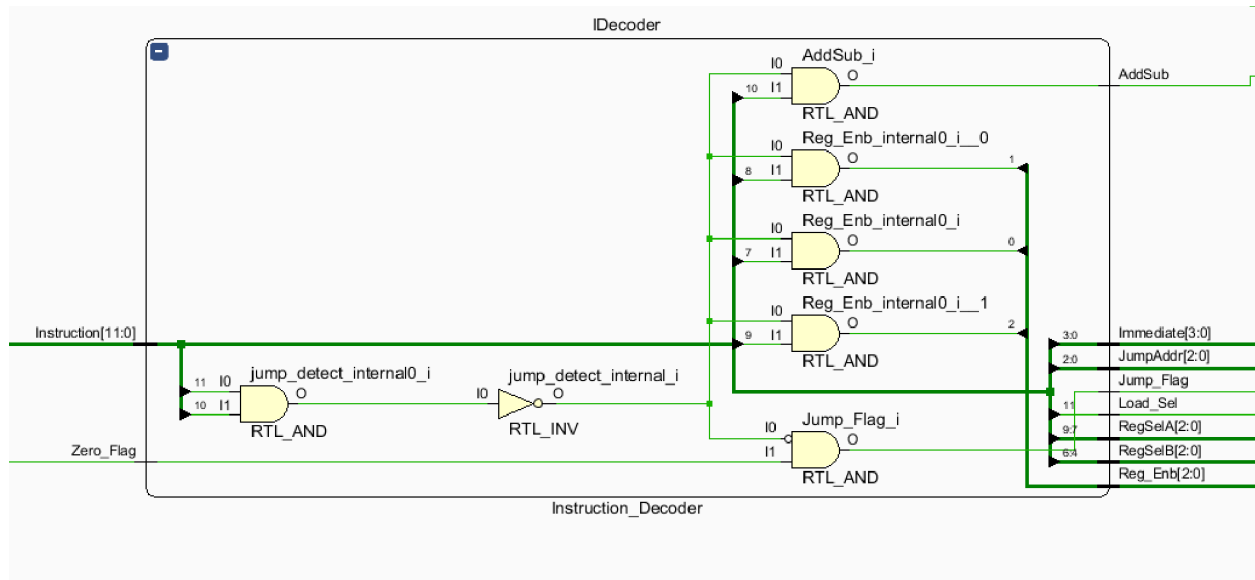
```

1 0 0 0 1 0 0 0 0 0 0 1      ; MOVI R1, 1
1 0 0 1 0 0 0 0 0 1 0 0      ; MOVI R2, 2
0 0 0 0 1 0 0 1 0 0 0 0      ; ADD R1, R2
1 0 0 1 1 0 0 0 0 1 1 0      ; MOVI R3, 3
0 0 0 0 1 1 0 0 0 0 0 0      ; ADD R1, R3
1 0 1 1 1 0 0 0 0 0 0 0      ; MOVI R7, 0
0 0 1 1 1 0 0 1 0 0 0 0      ; ADD R7, R1

```

Elaborated Design Schematics





VHDL Codes of Pro version

1. Top Level Integration

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity E_ntel_Processor_Pro is
    Port (
        Clk      : in  STD_LOGIC;
        Reset    : in  STD_LOGIC;
        Zero     : out STD_LOGIC;
        OverFlow : out STD_LOGIC;
        Jump     : out STD_LOGIC;
        SW       : in  STD_LOGIC; -- Switch for Reg7 or ALU Output
        Value_Out : out STD_LOGIC_VECTOR(3 downto 0);
        AN       : out STD_LOGIC_VECTOR(3 downto 0); -- Anodes
        SEG      : out STD_LOGIC_VECTOR(6 downto 0) -- Segments
    );
end E_ntel_Processor_Pro;

architecture Behavioral of E_ntel_Processor_Pro is

    -- Components

    component Nano_Processor is
        Port (
            Clk      : in  STD_LOGIC;
            Reset    : in  STD_LOGIC;
            Zero_Flag : out STD_LOGIC;

```



```

        Overflow_Flag : out STD_LOGIC;
        Jump_Flag     : out STD_LOGIC;
        ALU_Output, R7_Out : out STD_LOGIC_VECTOR(3 downto 0);
        Current_Address : out STD_LOGIC_VECTOR(2 downto 0);
        Next_Address   : out STD_LOGIC_VECTOR(2 downto 0)
    );
end component;

component Mux_2_Way_4_bit is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);
        B      : in  STD_LOGIC_VECTOR(3 downto 0);
        Sel     : in  STD_LOGIC;
        Output  : out STD_LOGIC_VECTOR(3 downto 0)
    );
end component;

component Display is
    Port (
        clk      : in STD_LOGIC;
        digit0   : in STD_LOGIC_VECTOR(3 downto 0);
        digit2   : in STD_LOGIC_VECTOR(2 downto 0);
        digit3   : in STD_LOGIC_VECTOR(2 downto 0);
        an       : out STD_LOGIC_VECTOR(3 downto 0);
        seg      : out STD_LOGIC_VECTOR(6 downto 0)
    );
end component;

-- Signals
signal Zero_internal, OverFlow_internal, Jump_internal : STD_LOGIC; --
for flags
    signal ALU_Output_internal, R7_internal, Selected_Output :
STD_LOGIC_VECTOR(3 downto 0);
    signal Current_Address_internal, Next_Address_internal :
STD_LOGIC_VECTOR(2 downto 0);

begin

    -- Instantiate NanoProcessor
    Processor: Nano_Processor
        port map (
            Clk => Clk,
            Reset => Reset,
            Zero_Flag => Zero_internal,
            Overflow_Flag => OverFlow_internal,
            Jump_Flag => Jump_internal,
            R7_Out => R7_internal,
            ALU_Output => ALU_Output_internal,
            Current_Address => Current_Address_internal,
            Next_Address => Next_Address_internal
        );

    -- Instantiate Mux_8_Way_4_Bit
    Mux_output: Mux_2_Way_4_bit
        port map (
            A => ALU_Output_internal,
            B => R7_internal,
            Sel => SW,
            Output => Selected_Output

```

```

    );

-- Instantiate AllSevenSegments
Display_4: Display
    port map (
        clk => Clk,
        digit0 => Selected_Output,
        digit2(2 downto 0) => Current_Address_internal,
        digit3(2 downto 0) => Next_Address_internal,
        an => AN,
        seg => SEG
    );

-- Flag Outputs
Zero      <= Zero_internal;
OverFlow  <= OverFlow_internal;
Jump      <= Jump_internal;
Value_Out <= Selected_Output;

end Behavioral;

```

2. Basic Processor

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Nano_Processor is
    Port (
        Clk          : in  STD_LOGIC;           -- Main input clock
        Reset         : in  STD_LOGIC;           -- Reset signal
        Zero_Flag     : out STD_LOGIC;           -- ALU Zero flag
        Overflow_Flag : out STD_LOGIC;           -- ALU Overflow flag
        Jump_Flag     : out STD_LOGIC;           -- Jump signal from
decoder
        ALU_Output,
        R7_Out        : out STD_LOGIC_VECTOR(3 downto 0); -- ALU result
& Reg_7 output
        Current_Address : out STD_LOGIC_VECTOR(2 downto 0); -- Current PC
        Next_Address    : out STD_LOGIC_VECTOR(2 downto 0); -- Next PC
    );
end Nano_Processor;

architecture Behavioral of Nano_Processor is

    -- ===== Components =====

    -- Clock Divider: Slows down clock for human observation
    component Slow_Clk is
        Port (
            Clk_in  : in  STD_LOGIC;
            Clk_out : out STD_LOGIC
        );
    end component;

    -- PC Increment Unit: Handles PC updates and jumps
    component PC_Increment_Unit is
        Port (
            Clk          : in  STD_LOGIC;

```

```

        Reset      : in  STD_LOGIC;
        JumpAddr   : in  STD_LOGIC_VECTOR(2 downto 0);
        Jump_Flag  : in  STD_LOGIC;
        MemorySel  : out STD_LOGIC_VECTOR(2 downto 0);
        Next_Add   : out STD_LOGIC_VECTOR(2 downto 0)
    );
end component;

-- Program ROM: Instruction memory (8 x 12-bit instructions)
component Program_ROM is
    Port (
        Address      : in  STD_LOGIC_VECTOR(2 downto 0);
        Instruction   : out STD_LOGIC_VECTOR(11 downto 0)
    );
end component;

-- Instruction Decoder: Converts 12-bit instruction to control signals
component Instruction_Decoder is
    Port (
        Instruction : in  STD_LOGIC_VECTOR(11 downto 0);
        Zero_Flag   : in  STD_LOGIC;
        RegSelA     : out STD_LOGIC_VECTOR(2 downto 0);
        RegSelB     : out STD_LOGIC_VECTOR(2 downto 0);
        Reg_Enb     : out STD_LOGIC_VECTOR(2 downto 0);
        Immediate   : out STD_LOGIC_VECTOR(3 downto 0);
        Load_Sel    : out STD_LOGIC;
        AddSub      : out STD_LOGIC;
        JumpAddr    : out STD_LOGIC_VECTOR(2 downto 0);
        Jump_Flag    : out STD_LOGIC
    );
end component;

-- Register Bank: 8 general-purpose 4-bit registers
component Reg_Bank is
    Port (
        Clk          : in  STD_LOGIC;
        Reset        : in  STD_LOGIC;
        Reg_Select    : in  STD_LOGIC_VECTOR(2 downto 0);
        Data_In       : in  STD_LOGIC_VECTOR(3 downto 0);
        R0, R1, R2, R3,
        R4, R5, R6, R7 : out STD_LOGIC_VECTOR(3 downto 0)
    );
end component;

-- MUX: Selects between ALU result and immediate value
component Mux_2_Way_4_bit is
    Port (
        A            : in  STD_LOGIC_VECTOR(3 downto 0);
        B            : in  STD_LOGIC_VECTOR(3 downto 0);
        Sel          : in  STD_LOGIC;
        Output       : out STD_LOGIC_VECTOR(3 downto 0)
    );
end component;

-- ALU: Performs addition/subtraction using two registers
component Arithmetic_Unit is
    Port (
        RegSelA      : in  STD_LOGIC_VECTOR(2 downto 0);
        RegSelB      : in  STD_LOGIC_VECTOR(2 downto 0);
        R0, R1, R2, R3,
        R4, R5, R6, R7 : in  STD_LOGIC_VECTOR(3 downto 0);
    );
end component;

```

```

        Cin          : in  STD_LOGIC;
        Sum           : out STD_LOGIC_VECTOR(3 downto 0);
        Overflow      : out STD_LOGIC;
        Zero          : out STD_LOGIC
    );
end component;

-- ===== Internal Signals =====
signal Internal_Clk      : std_logic;
signal PC_out, Next_Add_internal : std_logic_vector(2 downto 0);
signal Instruction       : std_logic_vector(11 downto 0);
signal RegSelA, RegSelB  : std_logic_vector(2 downto 0);
signal Immediate        : std_logic_vector(3 downto 0);
signal AddSub, LoadSel  : std_logic;
signal Reg_Enb, JumpAddress : std_logic_vector(2 downto 0);
signal R0, R1, R2, R3   : std_logic_vector(3 downto 0);
signal R4, R5, R6, R7   : std_logic_vector(3 downto 0);
signal ALU_Out, DataIn  : std_logic_vector(3 downto 0);
signal Zero, Overflow, Jumpflag : std_logic;

-- ===== Architecture Begin =====

begin

    -- Clock Divider Instantiation
    CLOCK : Slow_Clk
        port map (
            Clk_in  => Clk,
            Clk_out => Internal_Clk
        );

    -- Program Counter Instantiation
    PC_Inc : PC_Increment_Unit
        port map (
            Clk          => Internal_Clk,
            Reset        => Reset,
            JumpAddr     => JumpAddress,
            Jump_Flag    => Jumpflag,
            MemorySel    => PC_out,
            Next_Add     => Next_Add_internal
        );

    -- ROM Instantiation
    ROM : Program_ROM
        port map (
            Address      => PC_out,
            Instruction  => Instruction
        );

    -- Instruction Decoder Instantiation
    IDecoder : Instruction_Decoder
        port map (
            Instruction => Instruction,
            Zero_Flag  => Zero,
            RegSelA    => RegSelA,
            RegSelB    => RegSelB,
            Reg_Enb    => Reg_Enb,
            Immediate  => Immediate,
            Load_Sel   => LoadSel,
            AddSub     => AddSub,
            JumpAddr   => JumpAddress,

```

```

        Jump_Flag    => Jumpflag
    );

-- Register Bank Instantiation
RegBank : Reg_Bank
port map (
    Clk            => Internal_Clk,
    Reset          => Reset,
    Reg_Select     => Reg_Enb,
    Data_In        => DataIn,
    R0 => R0, R1 => R1, R2 => R2, R3 => R3,
    R4 => R4, R5 => R5, R6 => R6, R7 => R7
);

-- Data Mux Instantiation (ALU vs Immediate)
DataMux : Mux_2_Way_4_bit
port map (
    A      => Immediate,
    B      => ALU_Out,
    Sel    => LoadSel,
    Output => DataIn
);

-- ALU Instantiation
ALU : Arithmetic_Unit
port map (
    RegSelA => RegSelA,
    RegSelB => RegSelB,
    R0 => R0, R1 => R1, R2 => R2, R3 => R3,
    R4 => R4, R5 => R5, R6 => R6, R7 => R7,
    Cin    => AddSub,
    Sum    => ALU_Out,
    Zero   => Zero,
    Overflow => Overflow
);

-- Output Mappings
Zero_Flag    <= Zero;
Overflow_Flag <= Overflow;
Jump_Flag    <= Jumpflag;

R7_Out <= R7;

ALU_Output    <= ALU_Out;
Current_Address <= PC_out;
Next_Address  <= Next_Add_internal;

end Behavioral;

```

3. Slow Clock

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Slow_Clk is
    Port (
        Clk_in   : in  STD_LOGIC;
        Clk_out  : out STD_LOGIC
    );
end Slow_Clk;

architecture Behavioral of Slow_Clk is
    signal count : unsigned(25 downto 0) := (others => '0'); -- 26-bit counter
begin

    process (Clk_in)
    begin
        if rising_edge(Clk_in) then
            count <= count + 1;
        end if;
    end process;

    -- Use bit 25 as output (toggles every ~33.5 million ticks = ~0.67s period)
    Clk_out <= count(25); -- ~1.49 Hz square wave

end Behavioral;
```

4. PC Increment Unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PC_Increment_Unit is
    Port (
        Clk       : in  STD_LOGIC;
        Reset     : in  STD_LOGIC;
        JumpAddr  : in  STD_LOGIC_VECTOR(2 downto 0);
        Jump_Flag : in  STD_LOGIC;
        MemorySel : out STD_LOGIC_VECTOR(2 downto 0);
        Next_Add  : out STD_LOGIC_VECTOR(2 downto 0)
    );
end PC_Increment_Unit;

architecture Behavioral of PC_Increment_Unit is

    -- Component declarations
    component RCA_3
        Port (
            A      : in  STD_LOGIC_VECTOR(2 downto 0);
            B      : in  STD_LOGIC_VECTOR(2 downto 0);
            Sum    : out STD_LOGIC_VECTOR(2 downto 0);
            Cout   : out STD_LOGIC
        );
    end component;
```

```

end component;

component Mux_2_Way_3_bit
Port (
    A      : in  STD_LOGIC_VECTOR(2 downto 0);
    B      : in  STD_LOGIC_VECTOR(2 downto 0);
    Sel    : in  STD_LOGIC;
    Output : out STD_LOGIC_VECTOR(2 downto 0)
);
end component;

component Program_Counter
Port (
    Clk      : in  STD_LOGIC;
    Reset    : in  STD_LOGIC;
    PC_in    : in  STD_LOGIC_VECTOR(2 downto 0);
    PC_out   : out STD_LOGIC_VECTOR(2 downto 0)
);
end component;

-- Internal signals
signal PC_out_internal : STD_LOGIC_VECTOR(2 downto 0);
signal PC_plus_1       : STD_LOGIC_VECTOR(2 downto 0);
signal Next_PC         : STD_LOGIC_VECTOR(2 downto 0);

begin

    -- Instantiate Program Counter
    PC : Program_Counter
        port map (
            Clk      => Clk,
            Reset    => Reset,
            PC_in    => Next_PC,
            PC_out   => PC_out_internal
        );

    -- 3-bit RCA adds 1 to current PC
    ADDER : RCA_3
        port map (
            A      => PC_out_internal,
            B      => "001", -- Increment PC by 1
            Sum    => PC_plus_1,
            Cout   => open    -- Overflow unused
        );

    -- MUX selects between JumpAddr and PC+1 based on Jump_Flag
    MUX : Mux_2_Way_3_bit
        port map (
            A      => JumpAddr,
            B      => PC_plus_1,
            Sel    => Jump_Flag,
            Output => Next_PC
        );

    -- Output current PC and next PC values
    MemorySel <= PC_out_internal;
    Next_Add  <= Next_PC;

end Behavioral;

```

5. Program Counter

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Program_Counter is
    Port (
        Clk      : in  std_logic;           -- Clock signal
        Reset    : in  std_logic;           -- Asynchronous reset, active high
        PC_in    : in  std_logic_vector(2 downto 0); -- Input program
        counter value
        PC_out   : out std_logic_vector(2 downto 0) -- Current program
        counter output
    );
end Program_Counter;

architecture Behavioral of Program_Counter is
begin
    process(Clk, Reset)
    begin
        -- Asynchronous reset: reset PC to zero immediately when Reset='1'
        if Reset = '1' then
            PC_out <= "000";

            -- On rising edge of clock, update PC_out with PC_in
        elsif rising_edge(Clk) then
            PC_out <= PC_in;
        end if;
    end process;
end Behavioral;
```

6. 3-bit Adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RCA_3 is
    Port (
        A      : in  STD_LOGIC_VECTOR(2 downto 0); -- 3-bit input A
        B      : in  STD_LOGIC_VECTOR(2 downto 0); -- 3-bit input B
        Sum     : out STD_LOGIC_VECTOR(2 downto 0); -- 3-bit sum output
        Cout    : out STD_LOGIC                    -- Carry out
        (overflow)
    );
end RCA_3;

architecture Behavioral of RCA_3 is
    -- Full Adder component declaration for reuse
    component FA
        Port (
            A      : in  STD_LOGIC; -- Single bit input A
            B      : in  STD_LOGIC; -- Single bit input B
            C_in   : in  STD_LOGIC; -- Carry input
            S      : out STD_LOGIC; -- Sum output
            C_out  : out STD_LOGIC; -- Carry output
        );
    end component FA;
begin
    -- Full Adder component instantiation for reuse
    FA1 : FA
        port map (
            A => A(2), B => B(2), C_in => Cout,
            S => Sum(2), C_out => Cout
        );
    FA2 : FA
        port map (
            A => A(1), B => B(1), C_in => Cout,
            S => Sum(1), C_out => Cout
        );
    FA3 : FA
        port map (
            A => A(0), B => B(0), C_in => Cout,
            S => Sum(0), C_out => Cout
        );
end Behavioral;
```



```

    );
end component;

-- Internal carry signals between full adders
signal C0, C1 : STD_LOGIC;

begin

    -- First full adder: least significant bit addition with carry-in
    -- hardwired to '0'
    FA_0 : FA
    port map (
        A      => A(0),
        B      => B(0),
        C_in   => '0',      -- No carry-in for LSB adder
        S      => Sum(0),
        C_out  => C0        -- Carry out to next stage
    );

    -- Second full adder: middle bit addition, carry-in from previous
    -- adder
    FA_1 : FA
    port map (
        A      => A(1),
        B      => B(1),
        C_in   => C0,
        S      => Sum(1),
        C_out  => C1        -- Carry out to next stage
    );

    -- Third full adder: most significant bit addition, carry-out is
    -- overall overflow
    FA_2 : FA
    port map (
        A      => A(2),
        B      => B(2),
        C_in   => C1,
        S      => Sum(2),
        C_out  => Cout      -- Final carry out (overflow)
    );

end Behavioral;

```

7. Mux 2 way 3-bit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Mux_2_Way_3_bit is
    Port (
        A      : in  STD_LOGIC_VECTOR(2 downto 0);  -- Input A
        B      : in  STD_LOGIC_VECTOR(2 downto 0);  -- Input B
        Sel     : in  STD_LOGIC;                    -- Select signal
        Output  : out STD_LOGIC_VECTOR(2 downto 0)   -- Multiplexer output
    );
end Mux_2_Way_3_bit;

architecture Behavioral of Mux_2_Way_3_bit is

```

```

    -- Internal signals for selected inputs after gating with select lines
    signal SEL_A    : STD_LOGIC_VECTOR(2 downto 0);
    signal SEL_B    : STD_LOGIC_VECTOR(2 downto 0);
    signal NOT_SEL  : STD_LOGIC;
begin
    -- Invert select signal for gating input B
    NOT_SEL <= NOT Sel;

    -- Gate input A bits with Sel (passes when Sel='1')
    SEL_A(0) <= A(0) AND Sel;
    SEL_A(1) <= A(1) AND Sel;
    SEL_A(2) <= A(2) AND Sel;

    -- Gate input B bits with NOT Sel (passes when Sel='0')
    SEL_B(0) <= B(0) AND NOT_SEL;
    SEL_B(1) <= B(1) AND NOT_SEL;
    SEL_B(2) <= B(2) AND NOT_SEL;

    -- Combine gated signals to produce output bits
    Output(0) <= SEL_A(0) OR SEL_B(0);
    Output(1) <= SEL_A(1) OR SEL_B(1);
    Output(2) <= SEL_A(2) OR SEL_B(2);

end Behavioral;

```

8. Program ROM

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Program_ROM is
    Port (
        Address : in std_logic_vector(2 downto 0);
        Instruction : out std_logic_vector(11 downto 0)
    );
end Program_ROM;

architecture Behavioral of Program_ROM is
    type ROM_Array is array (0 to 7) of std_logic_vector(11 downto 0);

    -- Program to compute R0 = -3 + 5, conditionally negate, then add -2
    constant ROM_Data : ROM_Array := (
        -- "100010000001", -- MOVI R1, 1          ; R1 = 1
        -- "100100000010", -- MOVI R2, 2          ; R2 = 2 (for jump)
        -- "001110010000", -- ADD  R7, R1          ; R7 += R1 (R7 = 1)
        -- "000010010000", -- ADD  R1, R1          ; R1 += 1 (R1 = 2,
using R1 + R1 = 2)
        -- "010100000000", -- NEG  R2              ; R2 = -2
        -- "000100010000", -- ADD  R2, R1          ; R2 += R1 ? loop
control: -2 + 2 ? 0,
        -- "110100000010", -- JZR  R2              ; if R2 == 0, jump to
address 2 (line 3)
        -- "101110000110"  -- MOV  R7,6            ; Finally Show R7 = 6

        "101110000001", -- MOV  R7, 1            ; R7 = 1
        "100100001110", -- MOVI R2, -2            ; R2 = -2
        "010100000000", -- NEG  R2                ; R2 = -2
    );
end Behavioral;

```

```

        "001110100000", -- ADD R7, R2          ; R7 = 1 + (-2) = -1 ?
but we actually wanted 1 + 2 ? so fix this below
        "100010000011", -- MOVI R1, 3          ; R1 = 3
        "001110010000", -- ADD R7, R1          ; R7 += 3
        "110000000111", -- JZR R0, 7           ; Infinite loop (R0
always 0)
        "110000000110" -- JZR R0, 6           ; Infinite loop (R0
always 0) Jump Back and forth );
begin
    Instruction <= ROM_Data(to_integer(unsigned(Address)));
end Behavioral;

```

9. Reg Bank

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Reg_Bank is
    Port (
        Clk      : in STD_LOGIC;
        Reset    : in STD_LOGIC;
        Reg_Select : in STD_LOGIC_VECTOR (2 downto 0); -- 3-bit selector
        Data_In   : in STD_LOGIC_VECTOR (3 downto 0);
        R0        : out STD_LOGIC_VECTOR (3 downto 0);
        R1        : out STD_LOGIC_VECTOR (3 downto 0);
        R2        : out STD_LOGIC_VECTOR (3 downto 0);
        R3        : out STD_LOGIC_VECTOR (3 downto 0);
        R4        : out STD_LOGIC_VECTOR (3 downto 0);
        R5        : out STD_LOGIC_VECTOR (3 downto 0);
        R6        : out STD_LOGIC_VECTOR (3 downto 0);
        R7        : out STD_LOGIC_VECTOR (3 downto 0);
    );
end Reg_Bank;

architecture Behavioral of Reg_Bank is

    -- 4-bit register with no input (outputs 0)
    component Reg_Zero
        Port (
            Clk : in STD_LOGIC;
            Q  : out STD_LOGIC_VECTOR (3 downto 0)
        );
    end component;

    -- 4-bit register with enable and reset
    component Reg
        Port (
            D      : in STD_LOGIC_VECTOR (3 downto 0);
            En     : in STD_LOGIC;
            Clk    : in STD_LOGIC;
            Reset  : in STD_LOGIC;
            Q      : out STD_LOGIC_VECTOR (3 downto 0)
        );
    end component;

    -- 3-to-8 decoder component
    component Decoder_3_to_8
        Port (

```

```

        I : in STD_LOGIC_VECTOR (2 downto 0);
        Y : out STD_LOGIC_VECTOR (7 downto 0)
    );
end component;

-- Signal to hold the decoder output (individual write enables)
signal decoder_out : STD_LOGIC_VECTOR (7 downto 0);

begin

    -- Instantiate 3-to-8 decoder
    Decoder: Decoder_3_to_8
    port map (
        I => Reg_Select,
        Y => decoder_out
    );

    -- Register 0 is hardwired to zero
    Register_0: Reg_Zero
    port map (
        Clk => Clk,
        Q => R0
    );

    -- Remaining registers use decoder output as enable
    Register_1: Reg
    port map (
        D => Data_In,
        En => decoder_out(1),
        Clk => Clk,
        Reset => Reset,
        Q => R1
    );

    Register_2: Reg
    port map (
        D => Data_In,
        En => decoder_out(2),
        Clk => Clk,
        Reset => Reset,
        Q => R2
    );

    Register_3: Reg
    port map (
        D => Data_In,
        En => decoder_out(3),
        Clk => Clk,
        Reset => Reset,
        Q => R3
    );

    Register_4: Reg
    port map (
        D => Data_In,
        En => decoder_out(4),
        Clk => Clk,
        Reset => Reset,
        Q => R4
    );

```

```

Register_5: Reg
  port map (
    D => Data_In,
    En => decoder_out(5),
    Clk => Clk,
    Reset => Reset,
    Q => R5
  );

Register_6: Reg
  port map (
    D => Data_In,
    En => decoder_out(6),
    Clk => Clk,
    Reset => Reset,
    Q => R6
  );

Register_7: Reg
  port map (
    D => Data_In,
    En => decoder_out(7),
    Clk => Clk,
    Reset => Reset,
    Q => R7
  );

end Behavioral;

```

10. Mux 2 way 4-bit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Mux_2_Way_4_bit is
  Port (
    A      : in  STD_LOGIC_VECTOR(3 downto 0); -- Input A (4-bit)
    B      : in  STD_LOGIC_VECTOR(3 downto 0); -- Input B (4-bit)
    Sel     : in  STD_LOGIC;                  -- Select signal: when
    '1' select A, when '0' select B
    Output  : out STD_LOGIC_VECTOR(3 downto 0) -- Output (4-bit)
    reflecting selected input
  );
end Mux_2_Way_4_bit;

architecture Behavioral of Mux_2_Way_4_bit is
  -- Internal signals to hold gated inputs based on select line
  signal Sel_A   : STD_LOGIC_VECTOR(3 downto 0);
  signal Sel_B   : STD_LOGIC_VECTOR(3 downto 0);
  signal Not_Sel : STD_LOGIC;
begin
  -- Invert the select signal
  Not_Sel <= not Sel;

  -- Gate A inputs with Sel signal
  Sel_A(0) <= A(0) and Sel;
  Sel_A(1) <= A(1) and Sel;
  Sel_A(2) <= A(2) and Sel;
  Sel_A(3) <= A(3) and Sel;

```

```

-- Gate B inputs with inverted Sel signal
Sel_B(0) <= B(0) and Not_Sel;
Sel_B(1) <= B(1) and Not_Sel;
Sel_B(2) <= B(2) and Not_Sel;
Sel_B(3) <= B(3) and Not_Sel;

-- Combine gated inputs to produce output
Output(0) <= Sel_A(0) or Sel_B(0);
Output(1) <= Sel_A(1) or Sel_B(1);
Output(2) <= Sel_A(2) or Sel_B(2);
Output(3) <= Sel_A(3) or Sel_B(3);

end Behavioral;

```

11. Mux 8 way 4-bit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Mux_8_Way_4_bit is
    Port (
        R0, R1, R2, R3, R4, R5, R6, R7 : in STD_LOGIC_VECTOR(3 downto 0);
        RegSel      : in  STD_LOGIC_VECTOR(2 downto 0);
        Data_Out    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end Mux_8_Way_4_bit;

architecture Behavioral of Mux_8_Way_4_bit is

    -- Component declaration for the 3-to-8 decoder
    component Decoder_3_to_8
        Port (
            I : in  STD_LOGIC_VECTOR(2 downto 0);
            Y : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;

    -- Internal signal to hold the decoder output
    signal internal_decoder_out : STD_LOGIC_VECTOR(7 downto 0);

begin

    -- Instantiate the decoder
    Decoder_Internal: Decoder_3_to_8
        port map (
            I => RegSel,
            Y => internal_decoder_out
        );

    -- Multiplexer logic based on one-hot decoder output
    Data_Out <= R0 when internal_decoder_out(0) = '1' else
                R1 when internal_decoder_out(1) = '1' else
                R2 when internal_decoder_out(2) = '1' else
                R3 when internal_decoder_out(3) = '1' else
                R4 when internal_decoder_out(4) = '1' else
                R5 when internal_decoder_out(5) = '1' else
                R6 when internal_decoder_out(6) = '1' else
                R7;

end Behavioral;

```

12. ALU

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Arithmetic_Unit is
    Port (
        -- Selection signals for registers
        RegSelA    : in  STD_LOGIC_VECTOR(2 downto 0);
        RegSelB    : in  STD_LOGIC_VECTOR(2 downto 0);

        -- 8 register inputs (each 4 bits wide)
        R0, R1, R2, R3, R4, R5, R6, R7 : in STD_LOGIC_VECTOR(3 downto 0);

        -- Carry in signal (0: Add, 1: Subtract)
        Cin        : in  STD_LOGIC;

        -- Outputs
        Sum        : out STD_LOGIC_VECTOR(3 downto 0);
        Overflow    : out STD_LOGIC;
        Zero       : out STD_LOGIC
    );
end Arithmetic_Unit;

architecture Behavioral of Arithmetic_Unit is

    -- Component Declarations
    component Mux_8_Way_4_bit
        Port (
            R0, R1, R2, R3, R4, R5, R6, R7 : in STD_LOGIC_VECTOR(3 downto
0);
            RegSel    : in STD_LOGIC_VECTOR(2 downto 0);
            Data_Out   : out STD_LOGIC_VECTOR(3 downto 0)
        );
    end component;

    component Add_Sub_Unit
        Port (
            A          : in  STD_LOGIC_VECTOR(3 downto 0);
            B          : in  STD_LOGIC_VECTOR(3 downto 0);
            Cin        : in  STD_LOGIC;
            Sum        : out STD_LOGIC_VECTOR(3 downto 0);
            Overflow    : out STD_LOGIC;
            Zero       : out STD_LOGIC
        );
    end component;

    -- Intermediate signals for mux outputs
    signal A_mux_out : STD_LOGIC_VECTOR(3 downto 0);
    signal B_mux_out : STD_LOGIC_VECTOR(3 downto 0);

begin

    -- Instantiate Mux for A input
    MuxA: Mux_8_Way_4_bit
        port map (
            R0 => R0, R1 => R1, R2 => R2, R3 => R3,
            R4 => R4, R5 => R5, R6 => R6, R7 => R7,
            RegSel => RegSelA,
            Data_Out => A_mux_out
        );
```

```

-- Instantiate Mux for B input
MuxB: Mux_8_Way_4_bit
  port map (
    R0 => R0, R1 => R1, R2 => R2, R3 => R3,
    R4 => R4, R5 => R5, R6 => R6, R7 => R7,
    RegSel => RegSelB,
    Data_Out => B_mux_out
  );

-- Instantiate the Adder/Subtractor
Adder_Subtractor: Add_Sub_Unit
  port map (
    A      => A_mux_out,
    B      => B_mux_out,
    Cin    => Cin,
    Sum    => Sum,
    Overflow => Overflow,
    Zero   => Zero
  );

end Behavioral;

```

13. Add/Sub Unit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Entity for 4-bit Adder/Subtractor (signed)
entity Add_Sub_Unit is
  Port (
    A      : in  STD_LOGIC_VECTOR(3 downto 0); -- Operand A
    B      : in  STD_LOGIC_VECTOR(3 downto 0); -- Operand B
    Cin    : in  STD_LOGIC;                    -- Control: 0 = Add,
    1 = Subtract
    Sum    : out STD_LOGIC_VECTOR(3 downto 0); -- Result
    Overflow : out STD_LOGIC;                  -- Signed overflow
    flag
    Zero   : out STD_LOGIC                    -- Zero flag
  );
end Add_Sub_Unit;

architecture Behavioral of Add_Sub_Unit is

  -- Component declaration for 4-bit ripple carry adder
  component RCA_4
    Port (
      A      : in  STD_LOGIC_VECTOR(3 downto 0);
      B      : in  STD_LOGIC_VECTOR(3 downto 0);
      Cin    : in  STD_LOGIC;
      Sum    : out STD_LOGIC_VECTOR(3 downto 0);
      Cout   : out STD_LOGIC
    );
  end component;

  -- Internal signals
  signal A_xor : STD_LOGIC_VECTOR(3 downto 0); -- A XOR Cin
  signal S_temp : STD_LOGIC_VECTOR(3 downto 0); -- Temporary sum

```



```

    signal Cout      : STD_LOGIC;           -- Carry-out from
RCA
    signal Zero_temp : STD_LOGIC;           -- Temp signal for
zero detection

begin

    -- Compute A XOR Cin for subtraction (2's complement logic)
    A_xor(0) <= A(0) XOR Cin;
    A_xor(1) <= A(1) XOR Cin;
    A_xor(2) <= A(2) XOR Cin;
    A_xor(3) <= A(3) XOR Cin;

    -- Instantiate 4-bit RCA to compute B + A or B + ~A + 1
    RCA: RCA_4
        port map (
            A      => B,
            B      => A_xor,
            Cin    => Cin,
            Sum    => S_temp,
            Cout   => Cout
        );

    -- Drive final sum output
    Sum <= S_temp;

    -- Signed overflow detection: checks MSB behavior
    Overflow <= ((not B(3)) and (not A_xor(3)) and S_temp(3)) or
                (B(3) and A_xor(3) and (not S_temp(3)));

    -- Zero detection: check if all bits of result are 0
    Zero_temp <= S_temp(0) or S_temp(1) or S_temp(2) or S_temp(3);
    Zero <= not Zero_temp;

end Behavioral;

```

14. Display

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Display is
    Port ( clk : in STD_LOGIC;
          digit0 : in STD_LOGIC_VECTOR (3 downto 0);
          digit2 : in STD_LOGIC_VECTOR (2 downto 0);
          digit3 : in STD_LOGIC_VECTOR (2 downto 0);
          an : out STD_LOGIC_VECTOR (3 downto 0);
          seg : out STD_LOGIC_VECTOR (6 downto 0));
end Display;

architecture Behavioral of Display is
    -- Component declaration for the LUT
    component LUT_16_7 is
        Port (
            address : in STD_LOGIC_VECTOR (3 downto 0);
            data : out STD_LOGIC_VECTOR (6 downto 0)
        );
    end component;
end component;

```

```

-- constants
constant REFRESH_RATE : integer := 1000; -- 1kHz refresh rate
constant CLK_FREQ : integer := 100000000; -- 100MHz clock (adjust
according to your clock)
constant COUNTER_MAX : integer := CLK_FREQ / REFRESH_RATE / 4;

-- signals
signal refresh_counter : integer range 0 to COUNTER_MAX-1 := 0;
signal digit_value : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
signal digit_select : integer range 0 to 3 := 0;
signal newdigit0, newdigit1, newdigit2, newdigit3 : STD_LOGIC_VECTOR(3
downto 0);
signal seg_internal : STD_LOGIC_VECTOR(6 downto 0);

begin

    newdigit0 <= STD_LOGIC_VECTOR(unsigned(not digit0)+1) when digit0(3) =
'1' else digit0;
    newdigit2 <= "0" & digit2;
    newdigit3 <= "0" & digit3;

    -- Process for digit selection and refresh
    process(clk)
    begin
        if rising_edge(clk) then
            -- Refresh counter
            if refresh_counter = COUNTER_MAX-1 then
                refresh_counter <= 0;

                -- Move to next digit
                if digit_select = 3 then
                    digit_select <= 0;
                else
                    digit_select <= digit_select + 1;
                end if;
            else
                refresh_counter <= refresh_counter + 1;
            end if;
        end if;
    end process;

    -- Digit selection
    with digit_select select digit_value <=
        newdigit0 when 0,
        newdigit1 when 1,
        newdigit2 when 2,
        newdigit3 when 3;

    -- Instantiate the LUT for seven-segment decoding
    LUT : LUT_16_7
    port map (
        address => digit_value,
        data => seg_internal
    );

    -- Anode driver
    an <= "1110" when digit_select = 0 else
        "1101" when digit_select = 1 else
        "1011" when digit_select = 2 else
        "0111" when digit_select = 3 else

```

```

        "1111";

    -- Cathode driver
    seg <= "0111111" when (digit_select = 1 and digit0(3) = '1') else
        "1111111" when (digit_select = 1 and digit0(3) = '0') else
        seg_internal;

end Behavioral;

```

15. LUT 16 → 7

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity LUT_16_7 is
    Port ( address : in STD_LOGIC_VECTOR (3 downto 0);
          data : out STD_LOGIC_VECTOR (6 downto 0));
end LUT_16_7;

architecture Behavioral of LUT_16_7 is

    type rom_type is array (0 to 15) of std_logic_vector(6 downto 0);
    signal sevenSegment_ROM : rom_type := (

        "1000000", -- 0
        "1111001", -- 1
        "0100100", -- 2
        "0110000", -- 3
        "0011001", -- 4
        "0010010", -- 5
        "0000010", -- 6
        "1111000", -- 7
        "0000000", -- 8
        "0010000", -- 9
        "0001000", -- a
        "0000011", -- b
        "1000110", -- c
        "0100001", -- d
        "0000110", -- e
        "0001110" -- f
    );

begin

    data <= sevenSegment_ROM(to_integer(unsigned(address)));

end Behavioral;

```

VHDL Codes of Base version

1. Top Level Integration

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PC_Increment_Unit is
    Port (
        Clk      : in  STD_LOGIC;
        Reset    : in  STD_LOGIC;
        JumpAddr  : in  STD_LOGIC_VECTOR(2 downto 0);
        Jump_Flag : in  STD_LOGIC;
        MemorySel : out STD_LOGIC_VECTOR(2 downto 0)
    );
end PC_Increment_Unit;

architecture Behavioral of PC_Increment_Unit is

    -- Component declarations
    component RCA_3
        Port (
            A      : in  STD_LOGIC_VECTOR(2 downto 0);
            B      : in  STD_LOGIC_VECTOR(2 downto 0);
            Sum     : out STD_LOGIC_VECTOR(2 downto 0);
            Cout    : out STD_LOGIC
        );
    end component;

    component Mux_2_Way_3_bit
        Port (
            A      : in  STD_LOGIC_VECTOR(2 downto 0);
            B      : in  STD_LOGIC_VECTOR(2 downto 0);
            Sel     : in  STD_LOGIC;
            Output  : out STD_LOGIC_VECTOR(2 downto 0)
        );
    end component;

    component Program_Counter
        Port (
            Clk      : in  STD_LOGIC;
            Reset    : in  STD_LOGIC;
            PC_in    : in  STD_LOGIC_VECTOR(2 downto 0);
            PC_out   : out STD_LOGIC_VECTOR(2 downto 0)
        );
    end component;

    -- Internal signals
    signal PC_out_internal : STD_LOGIC_VECTOR(2 downto 0);
    signal PC_plus_1       : STD_LOGIC_VECTOR(2 downto 0);
    signal Next_PC         : STD_LOGIC_VECTOR(2 downto 0);

begin

    -- Instantiate Program Counter
    PC : Program_Counter
        port map (
            Clk      => Clk,
            Reset    => Reset,
            PC_in    => Next_PC,
```

```

        PC_out => PC_out_internal
    );

    -- 3-bit RCA adds 1 to current PC
    ADDER : RCA_3
    port map (
        A      => PC_out_internal,
        B      => "001", -- Increment PC by 1
        Sum    => PC_plus_1,
        Cout   => open    -- Overflow unused
    );

    -- MUX selects between JumpAddr and PC+1 based on Jump_Flag
    MUX : Mux_2_Way_3_bit
    port map (
        A      => JumpAddr,
        B      => PC_plus_1,
        Sel    => Jump_Flag,
        Output => Next_PC
    );

    -- Output current PC and next PC values
    MemorySel <= PC_out_internal;

end Behavioral;

```

2. PC Increment Unit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PC_Increment_Unit is
    Port (
        Clk      : in  STD_LOGIC;
        Reset    : in  STD_LOGIC;
        JumpAddr  : in  STD_LOGIC_VECTOR(2 downto 0);
        Jump_Flag : in  STD_LOGIC;
        MemorySel : out STD_LOGIC_VECTOR(2 downto 0)
    );
end PC_Increment_Unit;

architecture Behavioral of PC_Increment_Unit is

    -- Component declarations
    component RCA_3
    Port (
        A      : in  STD_LOGIC_VECTOR(2 downto 0);
        B      : in  STD_LOGIC_VECTOR(2 downto 0);
        Sum    : out STD_LOGIC_VECTOR(2 downto 0);
        Cout   : out STD_LOGIC
    );
    end component;

    component Mux_2_Way_3_bit
    Port (
        A      : in  STD_LOGIC_VECTOR(2 downto 0);
        B      : in  STD_LOGIC_VECTOR(2 downto 0);
        Sel    : in  STD_LOGIC;
        Output : out STD_LOGIC_VECTOR(2 downto 0)
    );
    end component;

```

```

    );
end component;

component Program_Counter
  Port (
    Clk      : in  STD_LOGIC;
    Reset    : in  STD_LOGIC;
    PC_in    : in  STD_LOGIC_VECTOR(2 downto 0);
    PC_out   : out STD_LOGIC_VECTOR(2 downto 0)
  );
end component;

-- Internal signals
signal PC_out_internal : STD_LOGIC_VECTOR(2 downto 0);
signal PC_plus_1       : STD_LOGIC_VECTOR(2 downto 0);
signal Next_PC         : STD_LOGIC_VECTOR(2 downto 0);

begin

  -- Instantiate Program Counter
  PC : Program_Counter
    port map (
      Clk      => Clk,
      Reset    => Reset,
      PC_in    => Next_PC,
      PC_out   => PC_out_internal
    );

  -- 3-bit RCA adds 1 to current PC
  ADDER : RCA_3
    port map (
      A      => PC_out_internal,
      B      => "001", -- Increment PC by 1
      Sum    => PC_plus_1,
      Cout   => open   -- Overflow unused
    );

  -- MUX selects between JumpAddr and PC+1 based on Jump_Flag
  MUX : Mux_2_Way_3_bit
    port map (
      A      => JumpAddr,
      B      => PC_plus_1,
      Sel    => Jump_Flag,
      Output => Next_PC
    );

  -- Output current PC and next PC values
  MemorySel <= PC_out_internal;

end Behavioral;

```

3. LUT 16 7

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity LUT_16_7 is
    Port ( address : in STD_LOGIC_VECTOR (3 downto 0);
          data : out STD_LOGIC_VECTOR (6 downto 0));
end LUT_16_7;

architecture Behavioral of LUT_16_7 is

    type rom_type is array (0 to 7) of std_logic_vector(6 downto 0);
    signal sevenSegment_ROM : rom_type := (
        "1000000", -- 0
        "1111001", -- 1
        "0100100", -- 2
        "0110000", -- 3
        "0011001", -- 4
        "0010010", -- 5
        "0000010", -- 6
        "1111000" -- 7
    );

begin
    data <= sevenSegment_ROM(to_integer(unsigned(address)));
end Behavioral;
```

Other VHDL files are same as Pro version

Simulation Source Files for Pro version

1.Top Level

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity E_ntel_Processor_Pro_TB is
end E_ntel_Processor_Pro_TB;

architecture Behavioral of E_ntel_Processor_Pro_TB is

    -- Component Declaration
    component E_ntel_Processor_Pro
        Port (
            Clk          : in  STD_LOGIC;
            Reset         : in  STD_LOGIC;
            Zero          : out STD_LOGIC;
            OverFlow      : out STD_LOGIC;
            Jump          : out STD_LOGIC;
            SW            : in  STD_LOGIC;
            Value_Out     : out STD_LOGIC_VECTOR(3 downto 0);
            AN             : out STD_LOGIC_VECTOR(3 downto 0);
            SEG           : out STD_LOGIC_VECTOR(6 downto 0)
        );
    end component;

    -- Signals for Testbench
    signal Clk_tb          : STD_LOGIC := '0';
    signal Reset_tb        : STD_LOGIC := '0';
    signal Zero_tb         : STD_LOGIC;
    signal OverFlow_tb     : STD_LOGIC;
    signal Jump_tb         : STD_LOGIC;
    signal SW_tb           : STD_LOGIC;
    signal Value_tb        : STD_LOGIC_VECTOR(3 downto 0);
    signal AN_tb           : STD_LOGIC_VECTOR(3 downto 0);
    signal SEG_tb          : STD_LOGIC_VECTOR(6 downto 0);

begin

    -- Instantiate Unit Under Test (UUT)
    UUT: E_ntel_Processor_Pro
        port map (
            Clk          => Clk_tb,
            Reset        => Reset_tb,
            Zero         => Zero_tb,
            OverFlow     => OverFlow_tb,
            Jump         => Jump_tb,
            SW           => SW_tb,
            Value_Out    => Value_tb,
            AN           => AN_tb,
            SEG          => SEG_tb
        );

    -- Clock Process: 10 ns period (100 MHz)
    clk_process : process
    begin
        while true loop
            Clk_tb <= '1';
            wait for 10 ns;
        end loop
    end process;
```



```

        Clk_tb <= '0';
        wait for 10 ns;
    end loop;
end process;

-- Stimulus Process
stim_proc: process
begin

    SW_tb <= '0';

    -- Initial Reset
    Reset_tb <= '1';
    wait for 20 ns;
    Reset_tb <= '0';

    -- Test Mux select = 0 (ALU Output)

    wait for 100 ns;

    -- Test Mux select = 1 (R7 Output)
    SW_tb <= '1';
    wait for 100 ns;

    -- Toggle back to ALU
    SW_tb <= '0';
    wait for 100 ns;

    -- Simulation end (or continue with more tests)
    wait;
end process;

end Behavioral;

```

2.Basic Processor

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Nano_Processor_TB is
end Nano_Processor_TB;

architecture Behavioral of Nano_Processor_TB is

    -- Component under test
    component Nano_Processor
        Port (
            Clk           : in  STD_LOGIC;
            Reset          : in  STD_LOGIC;
            Zero_Flag      : out  STD_LOGIC;
            Overflow_Flag  : out  STD_LOGIC;
            Jump_Flag      : out  STD_LOGIC;
            ALU_Output,
            R7_Out         : out  STD_LOGIC_VECTOR(3 downto 0)
        );
    end component;

    -- Signals to connect to UUT

```

```

    signal Clk          : STD_LOGIC := '0';
    signal Reset        : STD_LOGIC := '1';
    signal Zero_Flag    : STD_LOGIC := '0';
    signal Overflow_Flag : STD_LOGIC := '0';
    signal Jump_Flag    : STD_LOGIC := '0';
    signal ALU_Output, R7_Out : STD_LOGIC_VECTOR(3 downto 0) := (others =>
'0');

begin

    -- Instantiate UUT
    UUT: Nano_Processor port map (
        Clk => Clk,
        Reset => Reset,
        Zero_Flag => Zero_Flag,
        Overflow_Flag => Overflow_Flag,
        Jump_Flag => Jump_Flag,
        R7_Out => R7_Out,
        ALU_Output => ALU_Output
    );

    -- Clock process: 100 MHz (10 ns period)
    Clock_Process : process
    begin
        loop
            Clk <= '1';
            wait for 10 ns;
            Clk <= '0';
            wait for 10 ns;
        end loop;
    end process;

    -- Reset and simulation control process
    Stim_Proc: process
    begin
        -- Hold reset active for proper initialization
        Reset <= '1';
        wait for 50 ns;
        Reset <= '0';

        -- Allow processor to run
        wait for 500 ns;

        -- End simulation
        wait;
    end process;

end Behavioral;

```

3.Display

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Display_TB is
end Display_TB;

architecture Behavioral of Display_TB is

```

```

-- Component Declaration
component Display
  Port (
    clk : in STD_LOGIC;
    digit0 : in STD_LOGIC_VECTOR(3 downto 0);
    digit2 : in STD_LOGIC_VECTOR(2 downto 0);
    digit3 : in STD_LOGIC_VECTOR(2 downto 0);
    an : out STD_LOGIC_VECTOR(3 downto 0);
    seg : out STD_LOGIC_VECTOR(6 downto 0)
  );
end component;

-- Signals for test
signal clk_tb      : std_logic := '0';
signal digit0_tb   : std_logic_vector(3 downto 0);
signal digit2_tb   : std_logic_vector(2 downto 0) := (others => '0');
signal digit3_tb   : std_logic_vector(2 downto 0) := (others => '0');
signal an_tb       : std_logic_vector(3 downto 0);
signal seg_tb      : std_logic_vector(6 downto 0);

-- Clock period constant
constant clk_period : time := 10 ns;

begin

  -- Instantiate the UUT
  UUT: Display
    port map (
      clk      => clk_tb,
      digit0   => digit0_tb,
      digit2   => digit2_tb,
      digit3   => digit3_tb,
      an       => an_tb,
      seg      => seg_tb
    );

  -- Clock generation
  clk_process : process
  begin
    while true loop
      clk_tb <= '0';
      wait for clk_period / 2;
      clk_tb <= '1';
      wait for clk_period / 2;
    end loop;
  end process;

  -- Stimulus process
  stim_proc: process
  begin
    -- Case 1
    digit0_tb <= "1101"; -- -3 in 2's complement
    digit2_tb <= "110";
    digit3_tb <= "001";

    wait for 200 ns;

    -- Case 2
    digit0_tb <= "0111";
    digit2_tb <= "010";
  end process;
end;

```

```

        digit3_tb <= "011";

        wait for 200 ns;

        -- Case 3
        digit0_tb <= "0011";
        digit2_tb <= "111";
        digit3_tb <= "111";
        wait;

        wait;
    end process;

end Behavioral;

```

4. Instruction Decoder

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Instruction_Decoder_TB is
end Instruction_Decoder_TB;

architecture Behavioral of Instruction_Decoder_TB is

    -- Component Declaration
    component Instruction_Decoder
        Port ( Instruction : in STD_LOGIC_VECTOR (11 downto 0);
              Zero_Flag : in STD_LOGIC;
              RegSelA : out STD_LOGIC_VECTOR (2 downto 0);
              RegSelB : out STD_LOGIC_VECTOR (2 downto 0);
              Reg_Enb : out STD_LOGIC_VECTOR (2 downto 0);
              Immediate : out STD_LOGIC_VECTOR (3 downto 0);
              Load_Sel : out STD_LOGIC;
              AddSub : out STD_LOGIC;
              JumpAddr : out STD_LOGIC_VECTOR (2 downto 0);
              Jump_Flag : out STD_LOGIC);
    end component;

    -- Signals for UUT
    signal Instruction : STD_LOGIC_VECTOR(11 downto 0);
    signal Zero_Flag : STD_LOGIC := '0';
    signal RegSelA, RegSelB, Reg_Enb : STD_LOGIC_VECTOR(2 downto 0);
    signal Immediate : STD_LOGIC_VECTOR(3 downto 0);
    signal Load_Sel, AddSub, Jump_Flag : STD_LOGIC;
    signal JumpAddr : STD_LOGIC_VECTOR(2 downto 0);

begin

    -- Instantiate the Unit Under Test (UUT)
    UUT: Instruction_Decoder
        port map (
            Instruction => Instruction,
            Zero_Flag => Zero_Flag,
            RegSelA => RegSelA,
            RegSelB => RegSelB,
            Reg_Enb => Reg_Enb,
            Immediate => Immediate,

```

```

        Load_Sel => Load_Sel,
        AddSub => AddSub,
        JumpAddr => JumpAddr,
        Jump_Flag => Jump_Flag
    );

-- Test process
stim_proc: process
begin

    -- Test Case 1: MOVI R3, 9 ? 10_011_000_1001 = "100110001001"
    Instruction <= "100110001001";
    wait for 100 ns;

    -- Test Case 2: MOVI R7, 15 (-1 in two's compliment) ?
10_111_000_1111 = "101110001111"
    Instruction <= "101110001111";
    wait for 100 ns;

    -- Test Case 3: ADD R1, R2 ? 00_001_010_0000 = "000010100000"
    Instruction <= "000010100000";
    wait for 100 ns;

    -- Test Case 4: ADD R5, R6 ? 00_101_110_0000 = "001011100000"
    Instruction <= "001011100000";
    wait for 100 ns;

    -- Test Case 5: NEG R4 ? 01_100_000_0000 = "011000000000"
    Instruction <= "011000000000";
    wait for 100 ns;

    -- Test Case 6: NEG R0 ? 01_000_000_0000 = "010000000000"
    Instruction <= "010000000000";
    wait for 100 ns;

    -- Test Case 7: JZR R2, 4 ? 11_010_000_0100 = "110100000100"
    Instruction <= "110100000100";
    Zero_Flag <= '1';
    wait for 100 ns;

    -- Test Case 8: JZR R1, 7 ? 11_001_000_0111 = "110010000111"
    Instruction <= "110010000111";
    Zero_Flag <= '0';
    wait for 100 ns;

    -- Test Case 9: MOVI R0, 1 ? 10_000_000_0001 = "100000000001"
    Instruction <= "100000000001";
    wait for 100 ns;

    -- Test Case 10: ADD R3, R1 ? 00_011_001_0000 = "000110010000"
    Instruction <= "000110010000";
    wait for 100 ns;

    wait;
end process;

end Behavioral;

```

5.Arithmetic Unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Arithmetic_Unit_TB is
end Arithmetic_Unit_TB;

architecture Behavioral of Arithmetic_Unit_TB is

    -- Component declaration
    component Arithmetic_Unit
        Port (
            RegSelA    : in  STD_LOGIC_VECTOR(2 downto 0);
            RegSelB    : in  STD_LOGIC_VECTOR(2 downto 0);
            R0, R1, R2, R3,
            R4, R5, R6, R7 : in  STD_LOGIC_VECTOR(3 downto 0);
            Cin        : in  STD_LOGIC;
            Sum        : out STD_LOGIC_VECTOR(3 downto 0);
            Overflow    : out STD_LOGIC;
            Zero       : out STD_LOGIC
        );
    end component;

    -- Signal declarations
    signal RegSelA, RegSelB : STD_LOGIC_VECTOR(2 downto 0);
    signal R0 : STD_LOGIC_VECTOR(3 downto 0) := "0000"; -- R0 is always 0
    signal R1, R2, R3, R4, R5, R6, R7 : STD_LOGIC_VECTOR(3 downto 0);
    signal Cin : STD_LOGIC;
    signal Sum : STD_LOGIC_VECTOR(3 downto 0);
    signal Overflow : STD_LOGIC;
    signal Zero : STD_LOGIC;

begin

    -- Instantiate UUT
    UUT: Arithmetic_Unit
        port map (
            RegSelA => RegSelA,
            RegSelB => RegSelB,
            R0 => R0, R1 => R1, R2 => R2, R3 => R3,
            R4 => R4, R5 => R5, R6 => R6, R7 => R7,
            Cin => Cin,
            Sum => Sum,
            Overflow => Overflow,
            Zero => Zero
        );

    -- Test process
    stimulus: process
    begin
        -- Initialize all
        RegSelA <= "000";
        RegSelB <= "000";
        R0 <= "0000";
        R1 <= "0000";
        R2 <= "0000";
        R3 <= "0000";
        R4 <= "0000";
        R5 <= "0000";
        R6 <= "0000";
    end process
end Behavioral;
```

```

R7 <= "0000";
Cin <= '0';

--wait for 10 ns;
-- Initialize registers (2's complement representation)
R0 <= "0000"; -- Hardcoded to 0
R1 <= "0010"; -- +2
R2 <= "0011"; -- +3
R3 <= "0100"; -- +4
R4 <= "1011"; -- -5
R5 <= "1100"; -- -4
R6 <= "1111"; -- -1
R7 <= "0111"; -- +7

--wait for 100 ns;

-- TEST CASES
-- Format: RegSelA, RegSelB, Cin

-- 1. -4 + 3 = -1
RegSelA <= "010"; -- R2 = +3
RegSelB <= "101"; -- R5 = -4
Cin <= '0';
wait for 100 ns;

-- 2. -2 + -7 = -9 (overflow)
R6 <= "1110"; -- -2
R5 <= "1001"; -- -7
RegSelA <= "110"; -- R6 = -2
RegSelB <= "101"; -- R5 = -7
Cin <= '0';
wait for 100 ns;

-- 3. 7 + 7 = 14 (overflow)
RegSelA <= "111"; -- R7 = +7
RegSelB <= "111"; -- R7 = +7
Cin <= '0';
wait for 100 ns;

-- 4. -1 + 1 = 0
R6 <= "1111"; -- -1
R1 <= "0001"; -- +1
RegSelA <= "110"; -- R6 = -1
RegSelB <= "001"; -- R1 = +1
Cin <= '0';
wait for 100 ns;

-- 5. 3 - 4 = -1 (B-A)
RegSelA <= "011"; -- R3 = +4
RegSelB <= "010"; -- R2 = +3
Cin <= '1'; -- Subtraction
wait for 100 ns;

-- 6. -4 - 4 = -8
RegSelA <= "100"; -- R4 = -5 (change it below)
R4 <= "1100"; -- -4
R3 <= "0100"; -- +4
RegSelB <= "011"; -- R3 = +4
Cin <= '1';
wait for 100 ns;

```

```

-- 7. -8 + 1 = -7
R4 <= "1000";      -- -8
R1 <= "0001";      -- +1
RegSelA <= "100";  -- R4 = -8
RegSelB <= "001";  -- R1 = +1
Cin      <= '0';
wait for 100 ns;

-- 8. 7 - 0 = 7
RegSelA <= "000";  -- R0 = 0
RegSelB <= "111";  -- R7 = +7
Cin      <= '1';
wait for 100 ns;

-- 9. -3 - (-5) = 2
R4 <= "1011";      -- -5
R5 <= "1101";      -- -3
RegSelA <= "100";  -- R4 = -5
RegSelB <= "101";  -- R5 = -3
Cin      <= '1';
wait for 100 ns;

-- 10. -6 - (+2) = -8 (overflow)
R1 <= "0010";      -- +2
R5 <= "1010";      -- -6
RegSelA <= "001";  -- R1 = +2
RegSelB <= "101";  -- R5 = -6
Cin      <= '1';
wait for 100 ns;

wait;
end process;

end Behavioral;

```

6.Program ROM

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Program_ROM_TB is
end Program_ROM_TB;

architecture Behavioral of Program_ROM_TB is

    -- Component Declaration
    component Program_ROM
        Port (
            Address      : in  std_logic_vector(2 downto 0);
            Instruction   : out std_logic_vector(11 downto 0)
        );
    end component;

    -- Signal Declarations
    signal Address       : std_logic_vector(2 downto 0) := (others => '0');
    signal Instruction   : std_logic_vector(11 downto 0);

begin

```



```

-- Instantiate UUT
UUT: Program_ROM
    port map (
        Address    => Address,
        Instruction => Instruction
    );

-- Test Process
stim_proc: process
begin
    -- Loop through all 8 addresses (0 to 7)
    for i in 0 to 7 loop
        Address <= std_logic_vector(to_unsigned(i, 3));
        wait for 100 ns;
    end loop;

    wait;
end process;

end Behavioral;

```

7.Reg Bank

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Reg_Bank_TB is
end Reg_Bank_TB;

architecture Behavioral of Reg_Bank_TB is

    -- Component Declaration
    component Reg_Bank
        Port (
            Clk          : in STD_LOGIC;
            Reset         : in STD_LOGIC;
            Reg_Select    : in STD_LOGIC_VECTOR (2 downto 0);
            Data_In       : in STD_LOGIC_VECTOR (3 downto 0);
            R0            : out STD_LOGIC_VECTOR (3 downto 0);
            R1            : out STD_LOGIC_VECTOR (3 downto 0);
            R2            : out STD_LOGIC_VECTOR (3 downto 0);
            R3            : out STD_LOGIC_VECTOR (3 downto 0);
            R4            : out STD_LOGIC_VECTOR (3 downto 0);
            R5            : out STD_LOGIC_VECTOR (3 downto 0);
            R6            : out STD_LOGIC_VECTOR (3 downto 0);
            R7            : out STD_LOGIC_VECTOR (3 downto 0);
        );
    end component;

    -- Signals
    signal Clk          : STD_LOGIC := '0';
    signal Reset        : STD_LOGIC := '0';
    signal Reg_Select    : STD_LOGIC_VECTOR (2 downto 0) := (others => '0');
    signal Data_In       : STD_LOGIC_VECTOR (3 downto 0) := (others => '0');
    signal R0, R1, R2, R3, R4, R5, R6, R7 : STD_LOGIC_VECTOR (3 downto 0);

    constant CLK_PERIOD : time := 100 ns;

```

```
begin
```

```
-- Clock generation  
Clk_process : process  
begin
```

```
    Clk <= '0';  
    wait for CLK_PERIOD/2;  
    Clk <= '1';  
    wait for CLK_PERIOD/2;  
end process;
```

```
-- UUT instantiation  
UUT: Reg_Bank
```

```
    port map (  
        Clk => Clk,  
        Reset => Reset,  
        Reg_Select => Reg_Select,  
        Data_In => Data_In,  
        R0 => R0,  
        R1 => R1,  
        R2 => R2,  
        R3 => R3,  
        R4 => R4,  
        R5 => R5,  
        R6 => R6,  
        R7 => R7  
    );
```

```
-- Stimulus process
```

```
stim_proc: process  
begin
```

```
-- Test 1: Reset all  
Reset <= '1';  
wait for CLK_PERIOD;  
Reset <= '0';  
wait for CLK_PERIOD;
```

```
-- Test 2: Write 0101 to Register 1  
Reg_Select <= "001"; Data_In <= "0101";  
wait for CLK_PERIOD;
```

```
-- Test 3: Write 1010 to Register 2  
Reg_Select <= "010"; Data_In <= "1010";  
wait for CLK_PERIOD;
```

```
-- Test 4: Write 1111 to Register 3  
Reg_Select <= "011"; Data_In <= "1111";  
wait for CLK_PERIOD;
```

```
-- Test 5: Write 0001 to Register 4  
Reg_Select <= "100"; Data_In <= "0001";  
wait for CLK_PERIOD;
```

```
-- Test 6: Write 0010 to Register 5  
Reg_Select <= "101"; Data_In <= "0010";  
wait for CLK_PERIOD;
```

```
-- Test 7: Write 0011 to Register 6  
Reg_Select <= "110"; Data_In <= "0011";  
wait for CLK_PERIOD;
```

```

-- Test 8: Write 0100 to Register 7
Reg_Select <= "111"; Data_In <= "0100";
wait for CLK_PERIOD;

-- Test 9: Attempt to write to R0 (should not change)
Reg_Select <= "000"; Data_In <= "1111";
wait for CLK_PERIOD;

-- Test 10: Reset again
Reset <= '1';
wait for CLK_PERIOD;
Reset <= '0';
wait for CLK_PERIOD;

-- End simulation
wait;
end process;

end Behavioral;

```

8.Two way 4-bit Mux

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Mux_2_Way_4_bit_TB is
end Mux_2_Way_4_bit_TB;

architecture Behavioral of Mux_2_Way_4_bit_TB is

-- Component declaration
component Mux_2_Way_4_bit
  Port (
    A      : in  STD_LOGIC_VECTOR(3 downto 0);
    B      : in  STD_LOGIC_VECTOR(3 downto 0);
    Sel    : in  STD_LOGIC;
    Output : out STD_LOGIC_VECTOR(3 downto 0)
  );
end component;

-- Testbench signals
signal A_tb, B_tb, Output_tb : STD_LOGIC_VECTOR(3 downto 0);
signal Sel_tb : STD_LOGIC;

begin

-- Instantiate the Unit Under Test (UUT)
UUT: Mux_2_Way_4_bit
  port map (
    A      => A_tb,
    B      => B_tb,
    Sel    => Sel_tb,
    Output => Output_tb
  );

-- Stimulus process
process
begin
  -- Test 1: A = "0001", B = "1110", Sel = 0 => Output = B

```

```

A_tb <= "0001";
B_tb <= "1110";
Sel_tb <= '0';
wait for 100 ns;

-- Test 2: A = "0001", B = "1110", Sel = 1 => Output = A
Sel_tb <= '1';
wait for 100 ns;

-- Test 3: A = "1010", B = "0101", Sel = 1 => Output = A
A_tb <= "1010";
B_tb <= "0101";
Sel_tb <= '1';
wait for 100 ns;

-- Test 4: A = "1010", B = "0101", Sel = 0 => Output = B
Sel_tb <= '0';
wait for 100 ns;

-- Test 5: A = "1111", B = "0000", Sel = 1 => Output = A
A_tb <= "1111";
B_tb <= "0000";
Sel_tb <= '1';
wait for 100 ns;

-- Finish simulation
wait;
end process;
end Behavioral;

```

9.PC Increment Unit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity PC_Increment_Unit_TB is
end PC_Increment_Unit_TB;

architecture Behavioral of PC_Increment_Unit_TB is

    -- Component Declaration
    component PC_Increment_Unit
        Port (
            Clk          : in STD_LOGIC;
            Reset         : in STD_LOGIC;
            JumpAddr      : in STD_LOGIC_VECTOR (2 downto 0);
            Jump_Flag     : in STD_LOGIC;
            MemorySel     : out STD_LOGIC_VECTOR (2 downto 0);
            Next_Add      : out STD_LOGIC_VECTOR(2 downto 0)
        );
    end component;

    -- Signals
    signal Clk          : STD_LOGIC := '0';
    signal Reset        : STD_LOGIC := '0';
    signal JumpAddr     : STD_LOGIC_VECTOR(2 downto 0) := (others => '0');
    signal Jump_Flag    : STD_LOGIC := '0';

```

```

signal MemorySel   : STD_LOGIC_VECTOR(2 downto 0);
signal Next_Add    : STD_LOGIC_VECTOR(2 downto 0);

-- Clock period
constant CLK_PERIOD : time := 200 ns;

begin

-- Instantiate Unit Under Test (UUT)
UUT: PC_Increment_Unit
  Port map (
    Clk       => Clk,
    Reset     => Reset,
    JumpAddr  => JumpAddr,
    Jump_Flag => Jump_Flag,
    MemorySel => MemorySel,
    Next_Add  => Next_Add
  );

-- Clock generation
Clk_Process : process
begin
  while true loop
    Clk <= '0';
    wait for CLK_PERIOD / 2;
    Clk <= '1';
    wait for CLK_PERIOD / 2;
  end loop;
end process;

-- Stimulus process
Stim_Proc: process
begin
  -- Test Case 01: Apply reset
  Reset <= '1';
  wait for CLK_PERIOD;
  Reset <= '0';

  -- Test Case 02: PC should increment to 001
  wait for CLK_PERIOD;

  -- Test Case 03: PC should increment to 010
  wait for CLK_PERIOD;

  -- Test Case 04: Jump to 101
  JumpAddr <= "101";
  Jump_Flag <= '1';
  wait for CLK_PERIOD;

  -- Test Case 05: PC should increment to 110
  Jump_Flag <= '0';
  wait for CLK_PERIOD;

  -- Test Case 06: Jump to 011
  JumpAddr <= "011";
  Jump_Flag <= '1';
  wait for CLK_PERIOD;

  -- Test Case 07: Apply reset again
  Jump_Flag <= '0';
  Reset <= '1';

```

```

    wait for CLK_PERIOD;
    Reset <= '0';

    -- Test Case 08: PC should increment to 001
    wait for CLK_PERIOD;

    -- Test Case 09: PC should increment to 010
    wait for CLK_PERIOD;

    -- Test Case 10: PC should increment to 011
    wait for CLK_PERIOD;

    wait; -- Stop simulation
end process;
end Behavioral;

```

10.Slow Clock

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Slow_Clk_TB is
end Slow_Clk_TB;

architecture sim of Slow_Clk_TB is

    -- Component Declaration for the UUT
    component Slow_Clk
        Port (
            Clk_in  : in  STD_LOGIC;
            Clk_out : out STD_LOGIC);
    end component;

    -- Testbench signals
    signal TB_Clk_in  : STD_LOGIC := '0';
    signal TB_Clk_out : STD_LOGIC;

    constant CLK_PERIOD : time := 10 ns;

begin

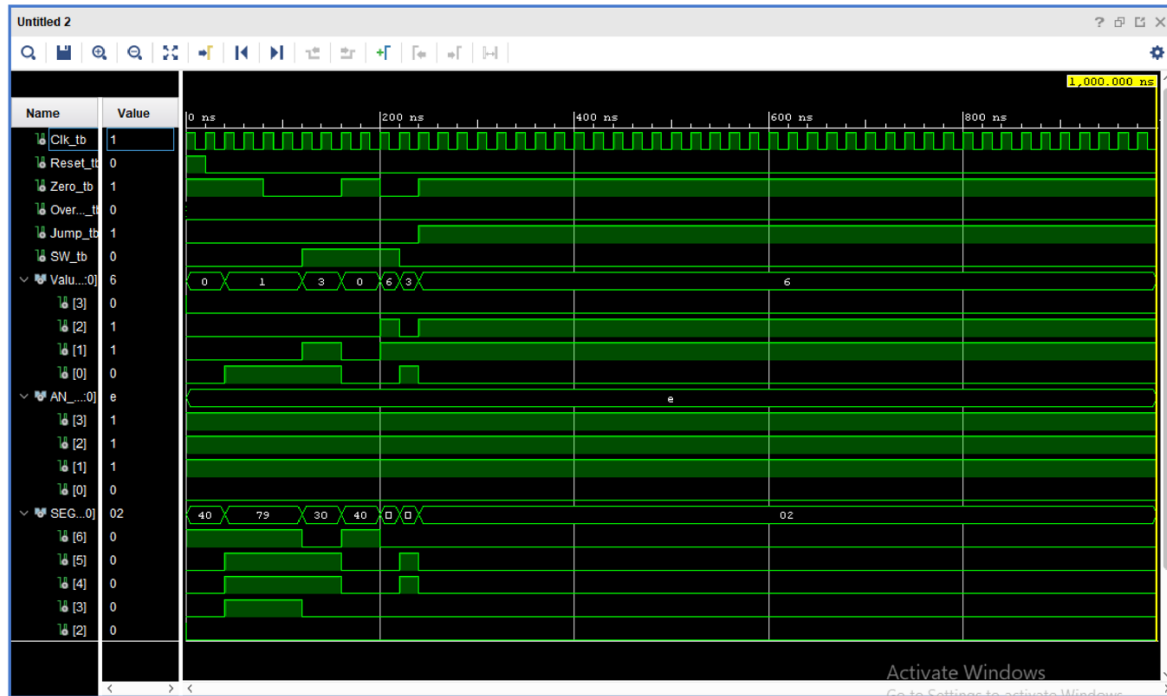
    -- Instantiate the Unit Under Test (UUT)
    UUT: Slow_Clk
        port map (
            Clk_in  => TB_Clk_in,
            Clk_out => TB_Clk_out);

    -- Generate 100 MHz clock
    clk_gen : process
    begin
        while now < 2 us loop -- Shortened for simulation speed
            TB_Clk_in <= '0';
            wait for CLK_PERIOD / 2;
            TB_Clk_in <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
        wait;
    end process;
end sim;

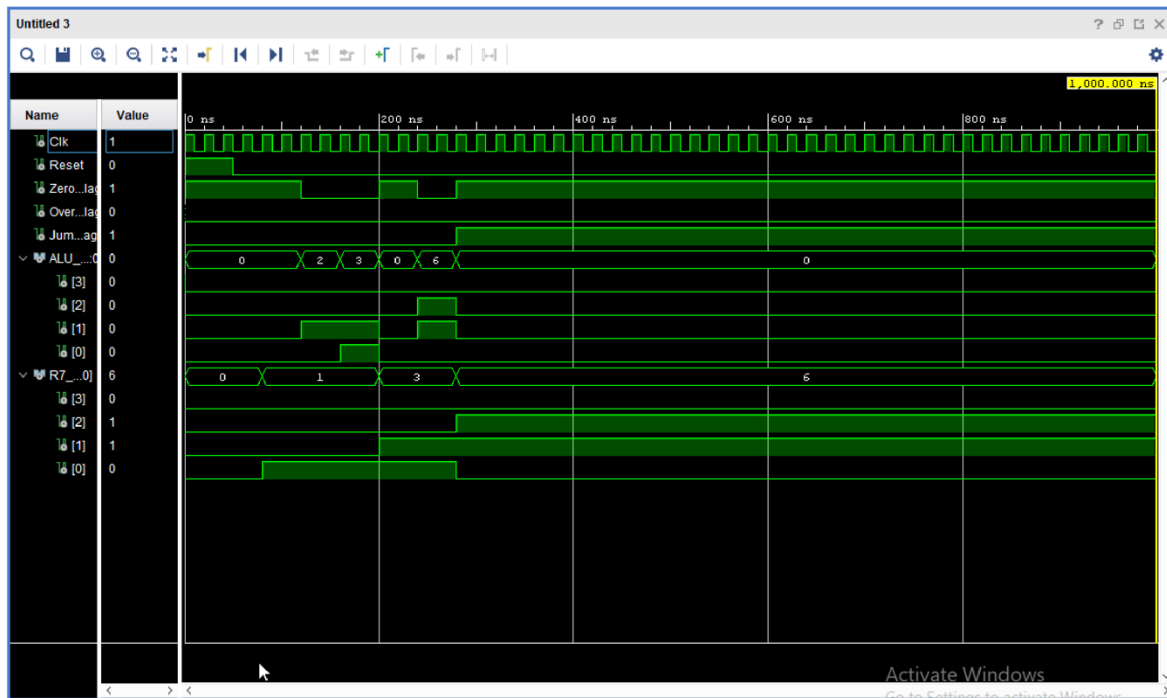
```

Timing Diagrams

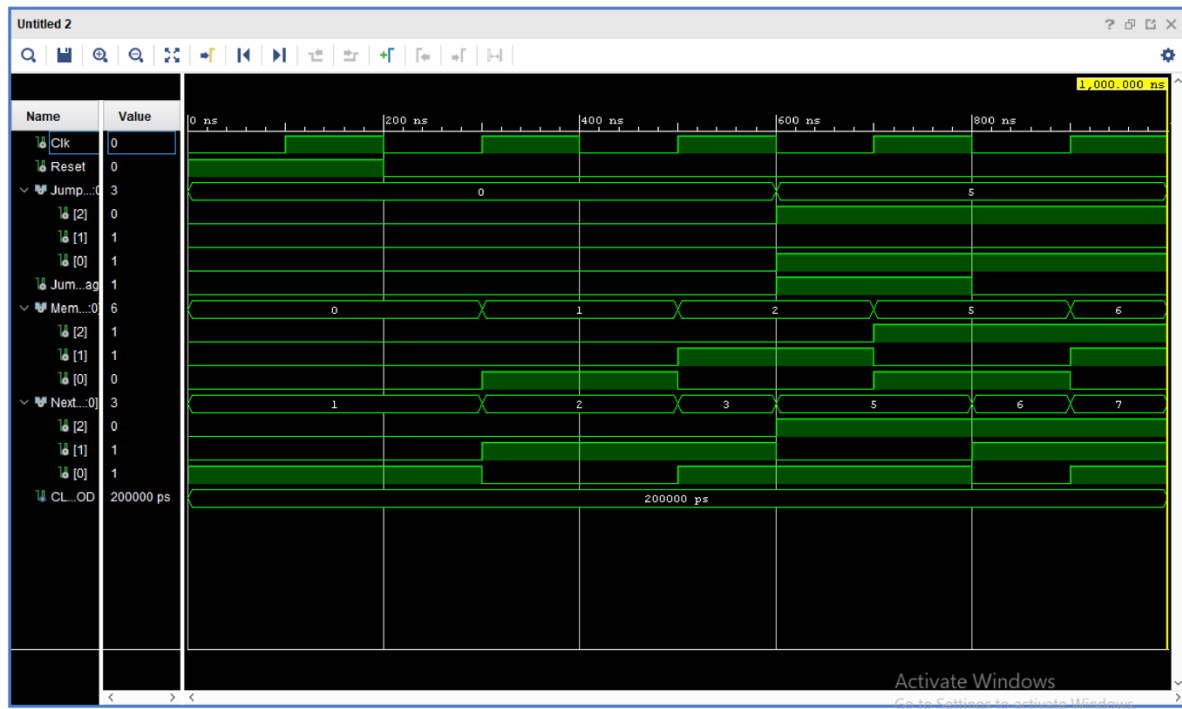
1.Top Level Integration



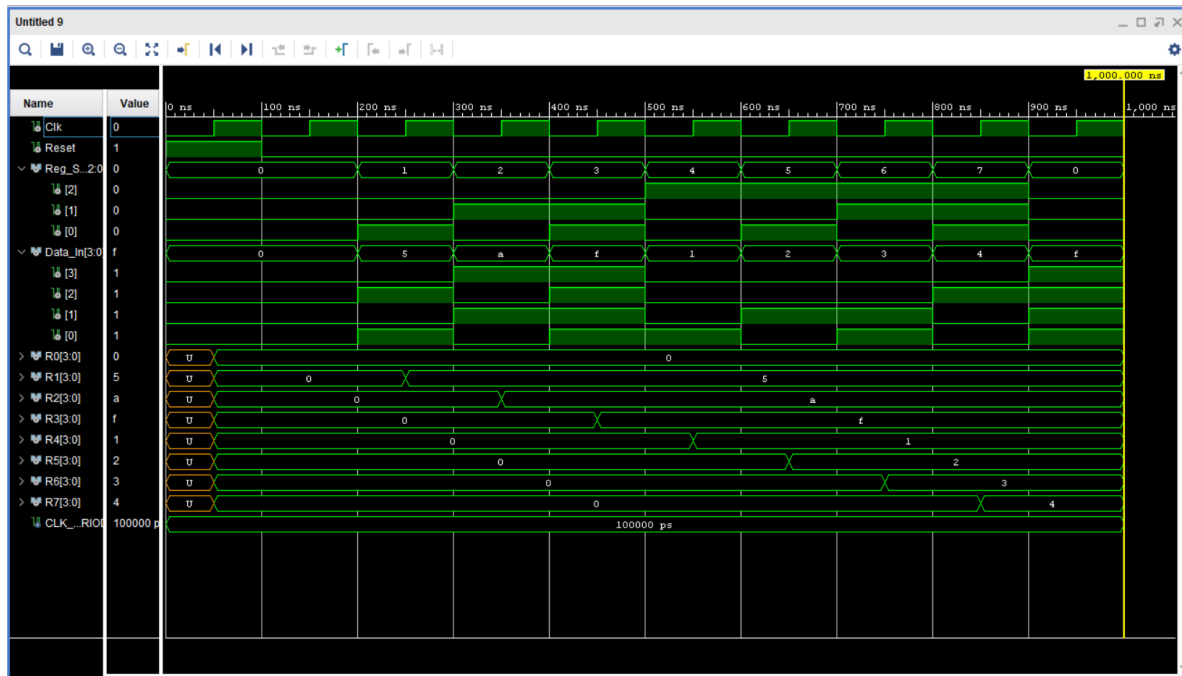
2.Basic Processor



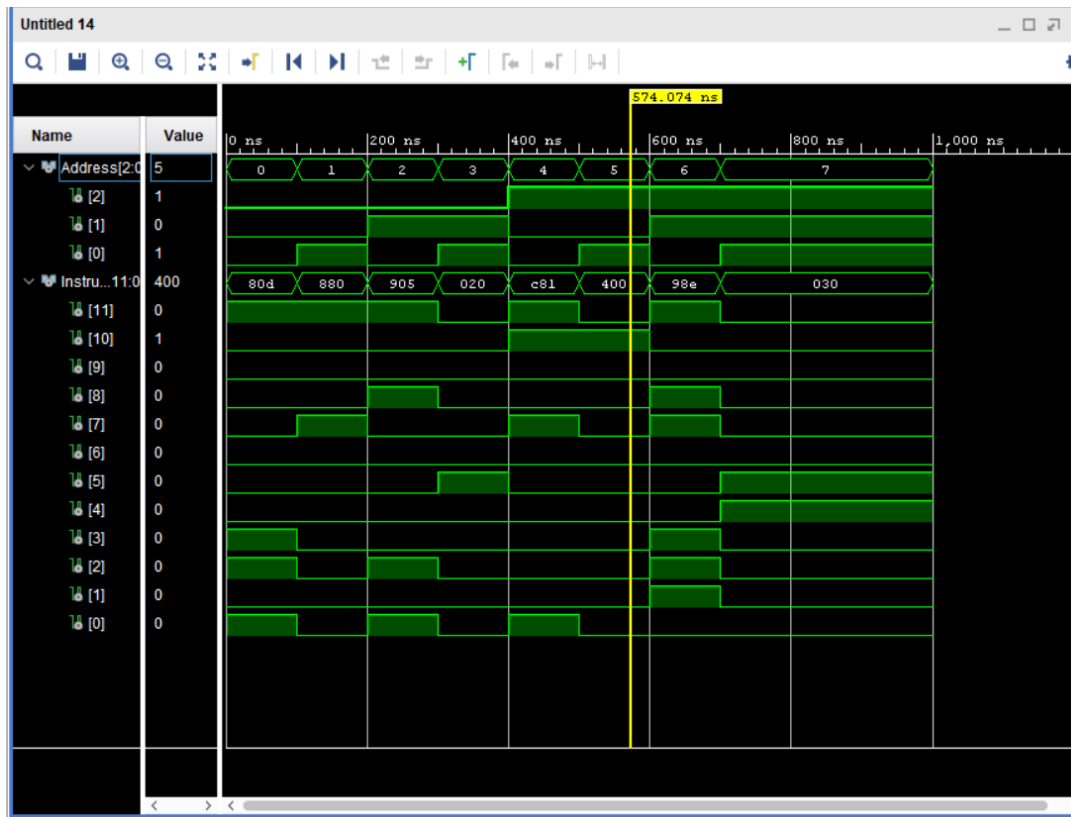
3. PC Increment Unit



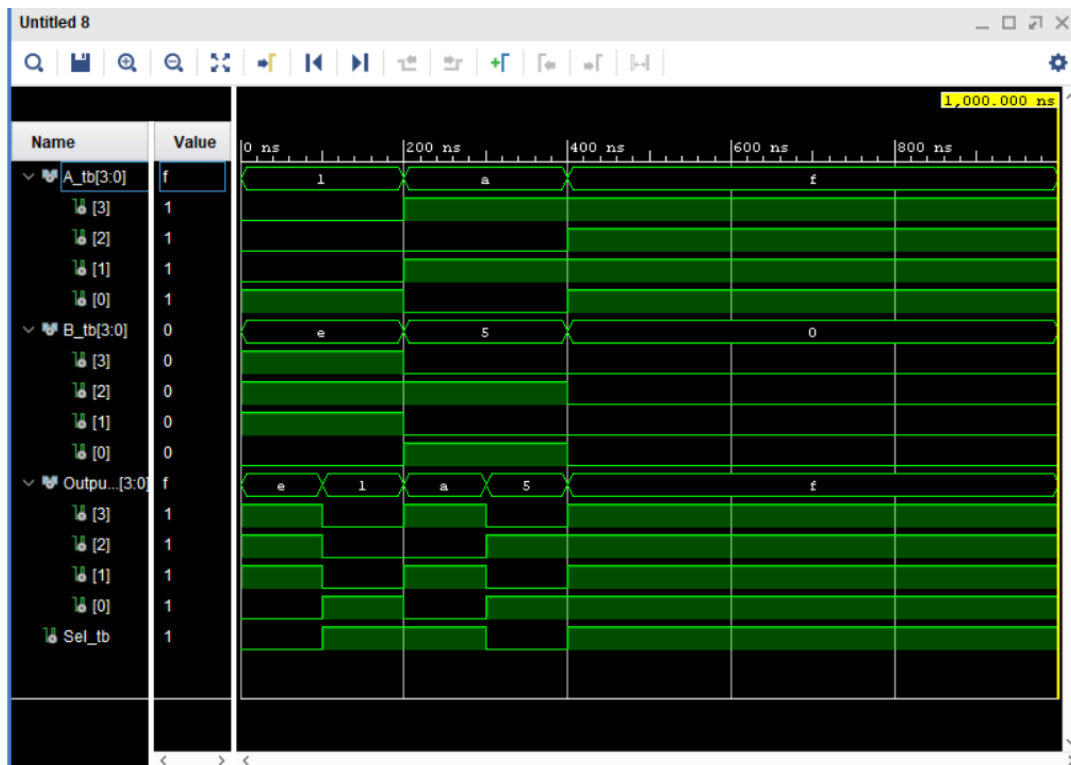
4. Reg Bank



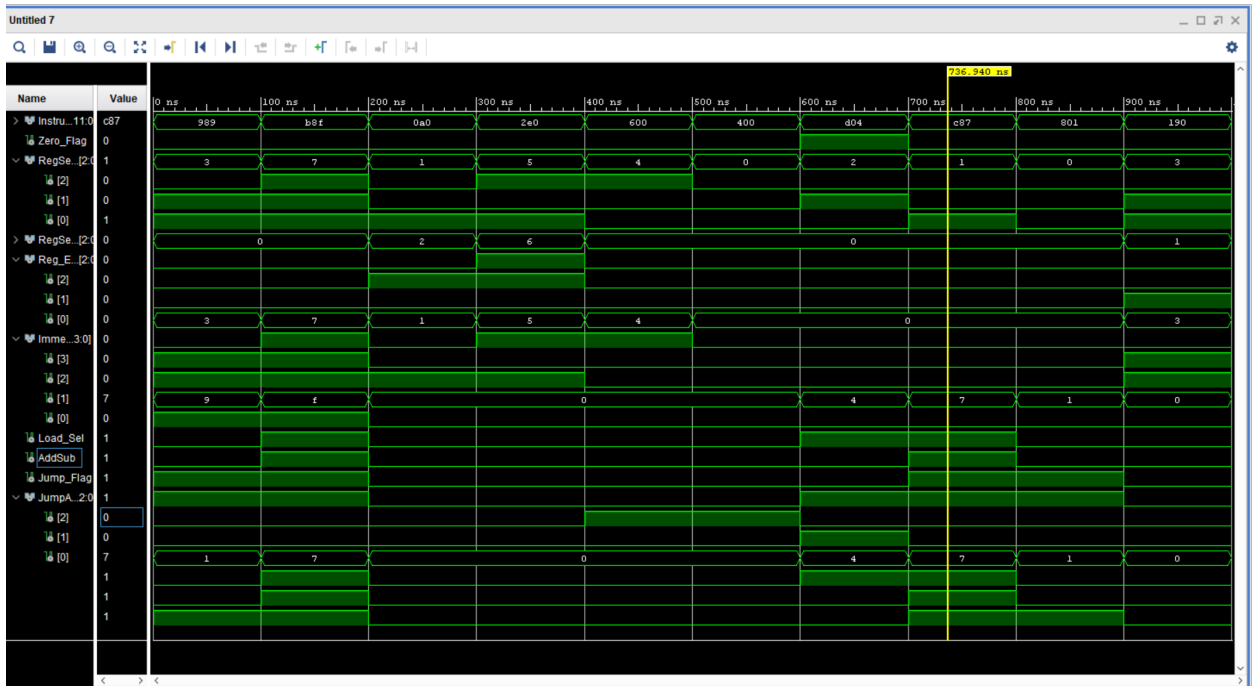
5. Program ROM



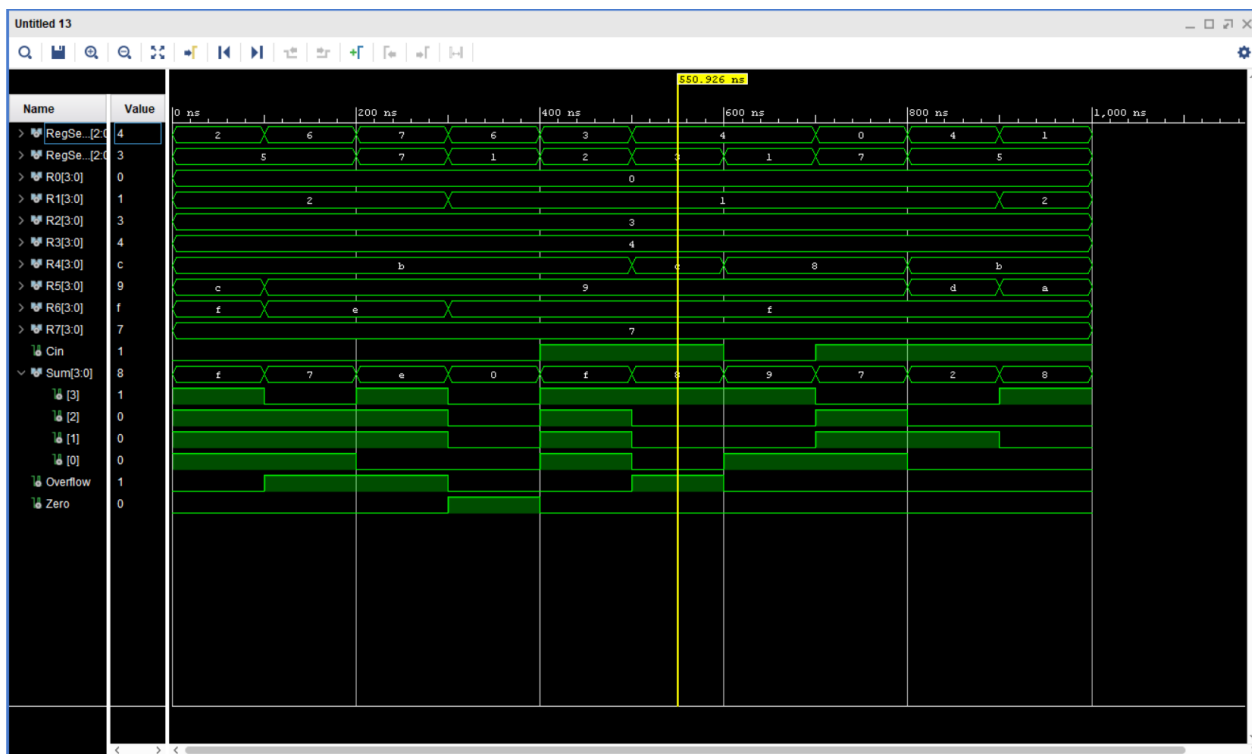
6. Mux



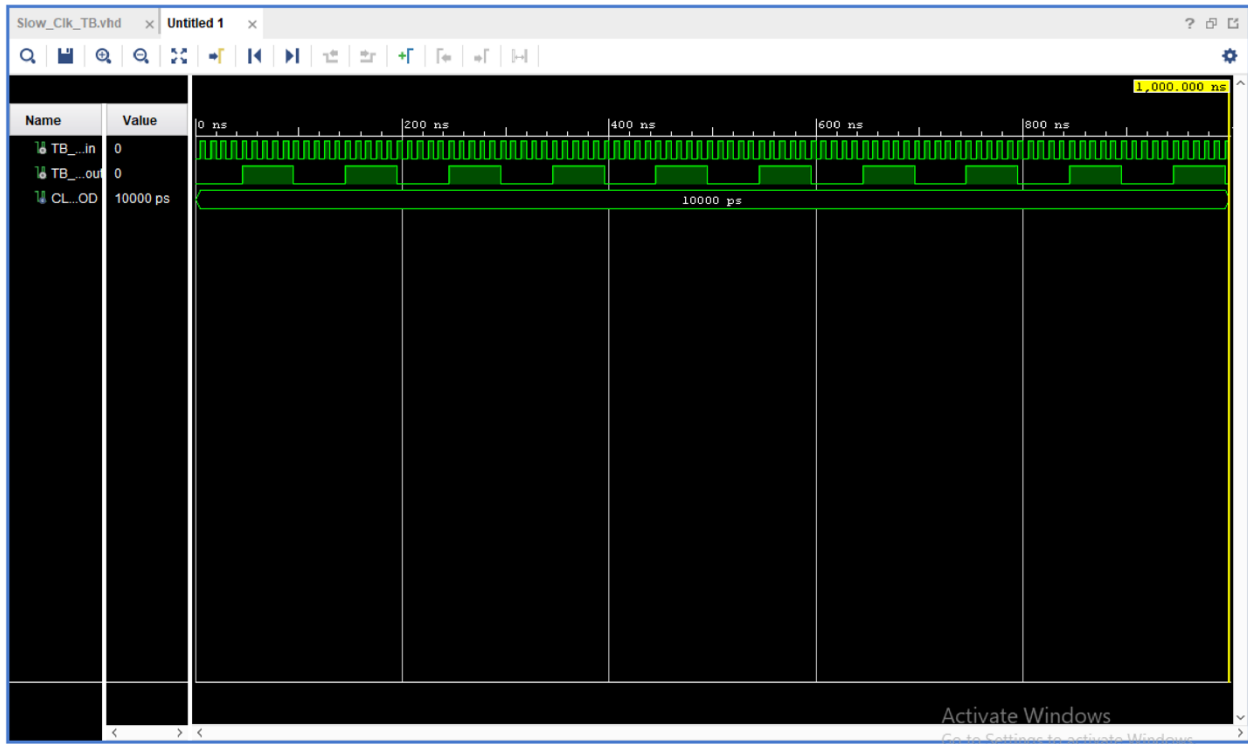
7. Instruction Decoder



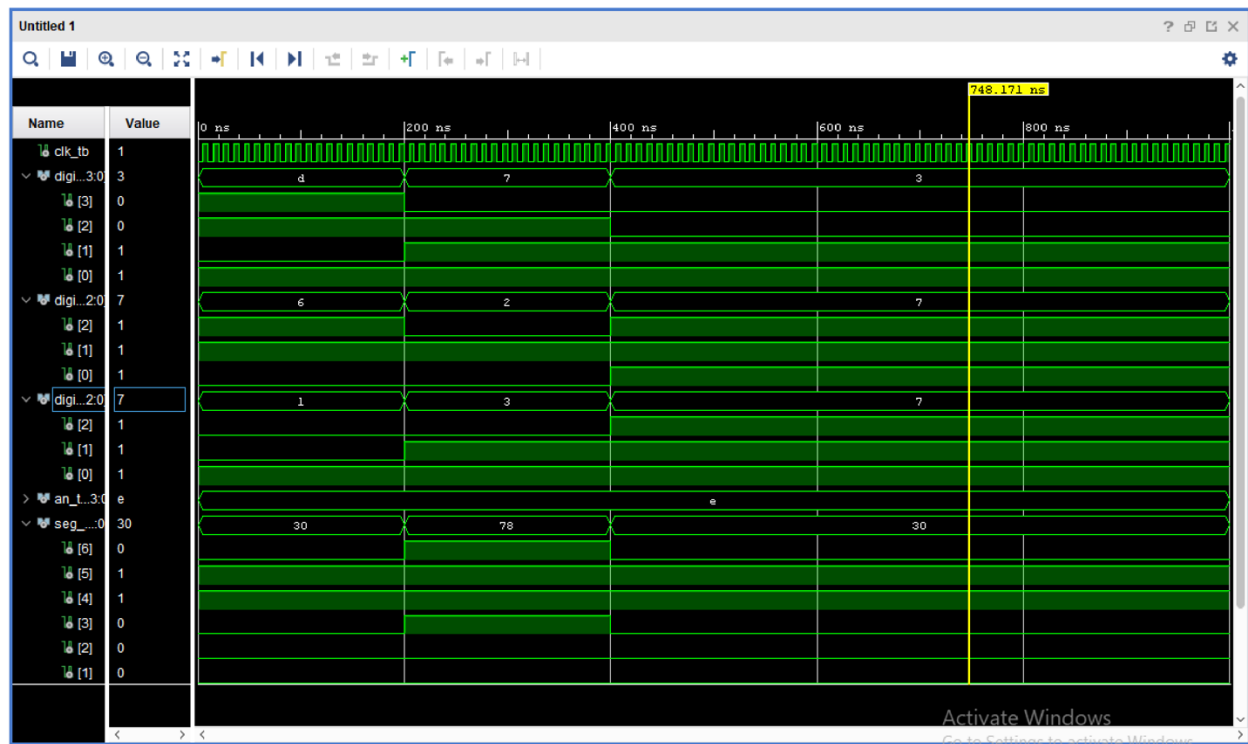
8.ALU



9.Slow Clock



10.Display



Constraint File

```
## Clock signal (Basys 3 onboard 100 MHz clock)
set_property PACKAGE_PIN W5 [get_ports Clk]
set_property IOSTANDARD LVCMOS33 [get_ports Clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
Clk]

## Switches for RegSel - SW0 is used
set_property PACKAGE_PIN V17 [get_ports {SW}]
set_property IOSTANDARD LVCMOS33 [get_ports {SW}]

## LEDs
# LD0 - LD3 = Value_Out
set_property PACKAGE_PIN U16 [get_ports {Value_Out[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Value_Out[0]}]
set_property PACKAGE_PIN E19 [get_ports {Value_Out[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Value_Out[1]}]
set_property PACKAGE_PIN U19 [get_ports {Value_Out[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Value_Out[2]}]
set_property PACKAGE_PIN V19 [get_ports {Value_Out[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Value_Out[3]}]

# LD13 = Jump
set_property PACKAGE_PIN P1 [get_ports Jump]
set_property IOSTANDARD LVCMOS33 [get_ports Jump]

# LD14 = Zero
set_property PACKAGE_PIN N3 [get_ports Zero]
set_property IOSTANDARD LVCMOS33 [get_ports Zero]

# LD15 = OverFlow
```

```
set_property PACKAGE_PIN L1 [get_ports OverFlow]
set_property IOSTANDARD LVCMOS33 [get_ports OverFlow]
```

7-segment display segments

```
set_property PACKAGE_PIN W7 [get_ports {SEG[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEG[0]}]
set_property PACKAGE_PIN W6 [get_ports {SEG[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEG[1]}]
set_property PACKAGE_PIN U8 [get_ports {SEG[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEG[2]}]
set_property PACKAGE_PIN V8 [get_ports {SEG[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEG[3]}]
set_property PACKAGE_PIN U5 [get_ports {SEG[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEG[4]}]
set_property PACKAGE_PIN V5 [get_ports {SEG[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEG[5]}]
set_property PACKAGE_PIN U7 [get_ports {SEG[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEG[6]}]
```

7-segment display anodes

```
set_property PACKAGE_PIN U2 [get_ports {AN[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[0]}]
set_property PACKAGE_PIN U4 [get_ports {AN[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[1]}]
set_property PACKAGE_PIN V4 [get_ports {AN[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[2]}]
set_property PACKAGE_PIN W4 [get_ports {AN[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[3]}]
```

Button for Reset - BTNC = U18

```
set_property PACKAGE_PIN U18 [get_ports Reset]
set_property IOSTANDARD LVCMOS33 [get_ports Reset]
```

Conclusions

- We successfully designed and implemented a simple 4-bit nanoprocessor capable of executing a limited set of instructions.
- The circuit was verified through simulation and demonstrated on the BASYS 3 board.
- We could visually observe register R7 holding the sum of values from 1 to 3 using LEDs.
- This lab enhanced our understanding of instruction decoding, register-based operations, and processor design principles.

Contributions and Time Spent

Team Member	Contribution	Hours Spent
Binara Herath	Top Level Integration/ Display/ Slow Clock	20 hrs
Janidu Pabasara	Instruction Decoder/ ALU	16 hrs
Malindu Bandara	ROM/ Muxes	13 hrs
Kalhara Jayathissa	PC Increment Unit/ Reg Bank	11 hrs